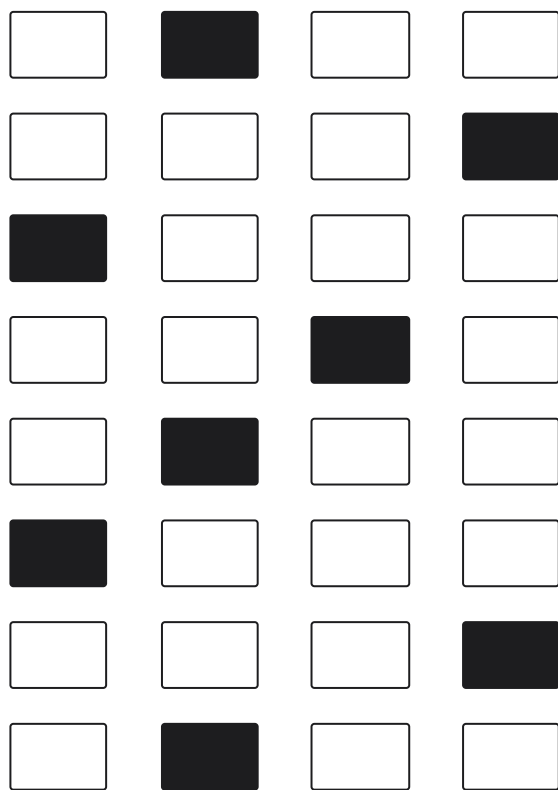




Corretor de gabarito

Apostila de estudo: da foto da folha de respostas até a nota, com a teoria de cada etapa e como ela foi aplicada no código.

Sumário

Antes de começar

1 Como executar

- 1.1 Pré-requisitos
- 1.2 Instalar o projeto (só uma vez)
- 1.3 Abrir o aplicativo
- 1.4 Usando o app
- 1.5 Imprimir e preencher a folha
- 1.6 Testar sem imprimir nada
- 1.7 Rodar os testes automatizados
- 1.8 Avaliar com fotos reais
- 1.9 Problemas comuns

2 Visão geral

- 2.1 O problema
- 2.2 A ideia central
- 2.3 O caminho completo
- 2.4 Mapa do código
- 2.5 Glossário rápido

3 Imagem digital

- Teoria
- Como aplicamos
- Para fixar

4 A folha de respostas

- Teoria: controlar a entrada simplifica o problema
- Como aplicamos

5 Marcadores ArUco: achando os cantos da folha

- Teoria
- Como aplicamos

6 Homografia: endireitando a folha

- Teoria
- Como aplicamos

7 Segmentação da tinta: separando tinta de papel

- Teoria
- Como aplicamos

8 Leitura das marcações

- Teoria
- Como aplicamos

9 OCR: lendo Nome, CPF e RG

- Teoria
- Como aplicamos

10 Correção e interface

- 10.1 O gabarito oficial
- 10.2 A nota
- 10.3 Como o Streamlit funciona
- 10.4 A tela de resultado
- 10.5 A imagem "grade detectada"
- 10.6 O visual

11 Testes e validação

11.1 TDD: o teste vem antes do código

11.2 O que é testado

11.3 Fotos sintéticas: como testar sem imprimir 500 folhas

11.4 Experimentos: provando que cada parte é necessária

11.5 Teste manual no navegador

11.6 Avaliação com amostras (scripts/avaliar.py)

11.7 O que ainda não foi provado

12 Como a IA foi usada

12.1 Ferramenta

12.2 O processo, em ordem

12.3 O que é do grupo (e não da IA)

12.4 Como conferir que o grupo entendeu

12.5 Mapa: conceito da disciplina → onde está no código

13 Perguntas e respostas

Sobre a abordagem

Sobre a leitura

Sobre o OCR

Sobre o software

A Roteiro da apresentação

B Resultados

C Avaliação das amostras sintéticas

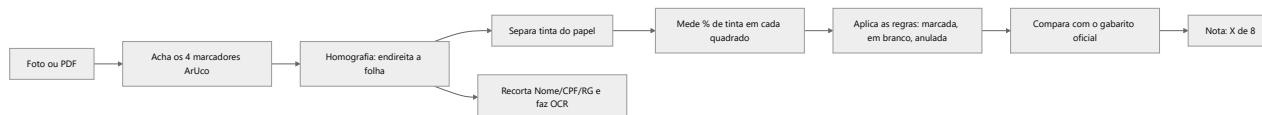
Antes de começar

Esta pasta explica **tudo** o que o software faz: primeiro a **teoria** de processamento de imagem por trás de cada etapa, depois **como ela foi aplicada** no código, com os valores reais que usamos e o motivo de cada escolha.

A ideia é que qualquer pessoa do grupo consiga ler na ordem, entender e explicar para a turma, inclusive a parte que foi feita com ajuda de IA.

■ **Versão para estudo em PDF:** [apostila.pdf](#) reúne todos os capítulos, com capa, sumário e apêndices. Para regerar depois de editar os .md: `.\.venv\Scripts\python.exe scripts\gerar_apostila.py`.

Resumo em uma frase por etapa



As figuras

Todas as imagens em `img/` são geradas pelo script `scripts/gerar_figuras_docs.py` a partir de uma foto de exemplo, usando **as mesmas funções do app**. Se alguém mudar o código, basta rodar o script de novo para atualizar as figuras:

```
.\.venv\Scripts\python.exe scripts\gerar_figuras_docs.py
```

Como executar

Tudo abaixo foi testado no **Windows 11** com **PowerShell**. Os comandos devem ser rodados dentro da pasta `trabalhoGabarito`.

1.1 Pré-requisitos

O quê	Por quê
Python 3.12	Linguagem do projeto
~2 GB livres em disco	O PyTorch (usado pelo OCR) é grande
Internet na primeira execução	Para instalar as bibliotecas e baixar os modelos do OCR

Instalar o Python

```
winget install Python.Python.3.12
```

Feche e abra o terminal depois de instalar. Para conferir:

```
py -3.12 --version
```

Armadilha comum no Windows: digitar só `python` pode abrir a Microsoft Store em vez do Python, porque o Windows tem um "atalho falso" chamado `python.exe`. Por isso usamos `py -3.12` na primeira vez e, depois, sempre o Python de dentro da `.venv`.

1.2 Instalar o projeto (só uma vez)

```
# 1. Criar um ambiente virtual (uma "caixa" com as bibliotecas só deste projeto)
py -3.12 -m venv .venv

# 2. Instalar as bibliotecas listadas em requirements.txt
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

O passo 2 demora alguns minutos. As bibliotecas instaladas são:

Biblioteca	Para que serve no projeto
<code>opencv-python-headless</code>	Processamento de imagem: ArUco, homografia, limiarização, contornos
<code>numpy</code>	Imagens são matrizes de números; o NumPy faz as contas
<code>pillow</code> + <code>pillow-heif</code>	Abrir JPG/PNG/HEIC (foto de iPhone) e desenhar a folha

Biblioteca	Para que serve no projeto
<code>pypdfium2</code>	Transformar a página de um PDF em imagem
<code>easyocr</code> (+ PyTorch)	Ler Nome, CPF e RG escritos à mão
<code>streamlit</code>	Interface web
<code>pytest</code>	Testes automatizados

Por que não ativamos a `.venv`? O comando `Activate.ps1` costuma ser bloqueado pela política de execução do PowerShell. Chamar `.\.venv\Scripts\python.exe` diretamente funciona sempre e tem o mesmo efeito.

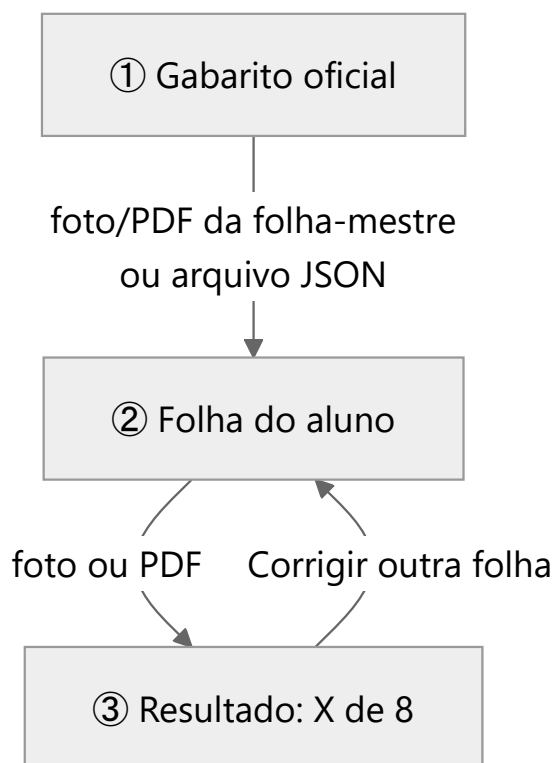
1.3 Abrir o aplicativo

```
.\.venv\Scripts\python.exe -m streamlit run app.py
```

O navegador abre sozinho em **`http://localhost:8501`**. Se não abrir, copie esse endereço. Para parar o app, aperte **`Ctrl + C`** no terminal.

 **A primeira correção demora ~20–30 segundos.** O modelo do OCR é carregado na memória na primeira folha de aluno (e baixado da internet na primeiríssima vez). As correções seguintes levam 1–3 segundos. **Antes de apresentar, corrija uma folha qualquer** para "aquecer" o app.

1.4 Usando o app



1. **Gabarito oficial:** escolha uma das formas: - **Foto ou PDF da folha-mestre:** uma folha preenchida com as respostas certas. Ela precisa ter **exatamente uma alternativa bem preenchida por questão**; se não tiver, o app recusa e diz quais questões conferir. - **Arquivo JSON:** um arquivo de texto assim: `json {"1": "A", "2": "C", "3": "B", "4": "D", "5": "A", "6": "A", "7": "C", "8": "B"}`
2. **Folha do aluno:** envie a foto (JPG, PNG, HEIC) ou o PDF.
3. **Resultado:** nota, Nome/CPF/RG lidos, a réplica da folha com cada questão colorida e, em **Ver processamento**, as imagens intermediárias (grade detectada, folha alinhada, arquivo original).

O gabarito oficial fica guardado enquanto a aba estiver aberta; dá para corrigir várias folhas seguidas. **Recarregar a página (F5) apaga o gabarito**, e ele precisa ser enviado de novo.

1.5 Imprimir e preencher a folha

O app tem o botão **Baixar folha para imprimir** no rodapé. Também dá para gerar pelo terminal:

```
.\.venv\Scripts\python.exe scripts\gerar_folha.py
```

Na impressão: - Papel **A4**. Pode imprimir em "tamanho real" ou "ajustar à página"; a mudança de escala é corrigida pelo software. - Confira se os **4 quadrados pretos dos cantos** saíram inteiros (sem corte da margem da impressora).

No preenchimento: - Caneta **preta ou azul**. Lápis ou caneta muito clara podem não ser detectados. - **Preencher o quadrado inteiro**. X, tracinho ou meio preenchido fazem a questão ser anulada (ou lida como em branco). - Escrever Nome, CPF e RG **dentro** das caixas.

Na foto: - Folha inteira aparecendo, com os 4 marcadores visíveis. Pode estar torta, girada ou de cabeça para baixo. - Evite reflexo forte (flash) em cima das marcações.

Escaneando em PDF: qualquer app de scanner (do celular ou da impressora) serve. O software usa **só a primeira página** do PDF.

1.6 Testar sem imprimir nada

```
.\.venv\Scripts\python.exe scripts\gerar_amstras.py
```

Isso cria em `amostras/sinteticas/` fotos simuladas (tortas, com sombra, de cabeça para baixo, um PDF) e o arquivo `gabarito_oficial.json`. É só enviar esses arquivos no app.

1.7 Rodar os testes automatizados

```
.\.venv\Scripts\python.exe -m pytest           # ~7 segundos
.\.venv\Scripts\python.exe -m pytest -m ocr     # teste do OCR real (mais lento)
```

O resultado esperado é algo como `88 passed, 1 deselected` (o "deselected" é o teste do OCR, que só roda com `-m ocr`).

Para ver **por que** cada parte do algoritmo existe, desligue-a e veja quais testes quebram (nada é alterado nos arquivos):

```
.\.venv\Scripts\python.exe scripts\experimentos.py           # lista
.\.venv\Scripts\python.exe scripts\experimentos.py sem_refinamento # exemplo
```

Mais detalhes no [capítulo 11](#).

1.8 Avaliar com fotos reais

1. Coloque as fotos/PDFs em `amostras/reais/`.
2. Para cada arquivo `x.jpg`, crie `x.json` com o que deveria ser lido: `json {"respostas": {"1": "A", "2": "ANULADA_MULTIPLA", "3": "EM_BRANCO", "4": "C", "5": "D", "6": "B", "7": "ANULADA_AMBIGUA", "8": "A"}, "nome": "Maria Eduarda Lima", "cpf": "987.654.321-00", "rg": "98.765.432-1"}`
3. Rode: powershell `.\.venv\Scripts\python.exe scripts\avaliar.py amostras\reais --saida docs\resultados_reais.md`

1.9 Problemas comuns

Sintoma	Causa provável	Solução
<code>python</code> abre a Microsoft Store	Atalho falso do Windows	Use <code>py -3.12</code> ou <code>.\.venv\Scripts\python.exe</code>
"Não encontrei os cantos ... da folha"	Foto cortada, marcador coberto/amassado, ou folha que não é a do software (ex.: a folha original do professor, que não tem marcadores)	Refaça a foto com a folha inteira; use a folha gerada pelo app
"A folha-mestre precisa de exatamente uma alternativa..."	Alguma questão da folha-mestre está em branco, dupla ou mal preenchida	Abra Ver processamento para ver a grade e corrija a folha
Mudei o código e o app continua igual	O Streamlit mantém os módulos antigos na memória	Pare com <code>Ctrl + C</code> e rode o app de novo
<code>Port 8501 is already in use</code>	O app já está aberto em outro terminal	Feche o outro ou use <code>--server.port 8502</code>
Primeira correção muito lenta	Carregando o modelo do OCR	Normal; as próximas são rápidas
Nome lido errado, "confiança baixa"	Limitação do OCR com letra de mão (principalmente cursiva)	Esperado; veja o capítulo 9
Marcação feita a lápis não aparece	Traço claro demais	Use caneta; ou ajuste os limites (capítulo 8)
Erro ao instalar o <code>easyocr</code> / <code>torch</code>	Internet caiu ou pouco espaço	Rode o <code>pip install</code> de novo

Visão geral

2.1 O problema

O enunciado do professor:

Fazer um software para ler os gabaritos de uma prova. A prova terá **8 questões**; cada questão terá **4 alternativas**; o software deverá dizer **quantas questões foram respondidas corretamente**.

Requisitos combinados depois:

Requisito	Onde é tratado
Anular questão com mais de uma alternativa marcada	Capítulo 8
Pedir que o quadrado seja todo preenchido	Instrução impressa na folha (cap. 4) + regra de "marcação incompleta" (cap. 8)
Ler o nome em letra de forma e em cursiva	Capítulo 9
Funcionar com impressões tortas / quadrados de tamanhos diferentes	Capítulos 5 , 6 e 8
Apresentar funcionando e dizer o que deu certo e o que não deu	apresentacao.md , resultados.md

Este tipo de sistema tem nome: **OMR** (*Optical Mark Recognition*, reconhecimento óptico de marcas). É a mesma ideia das folhas de resposta do ENEM e de concursos.

2.2 A ideia central

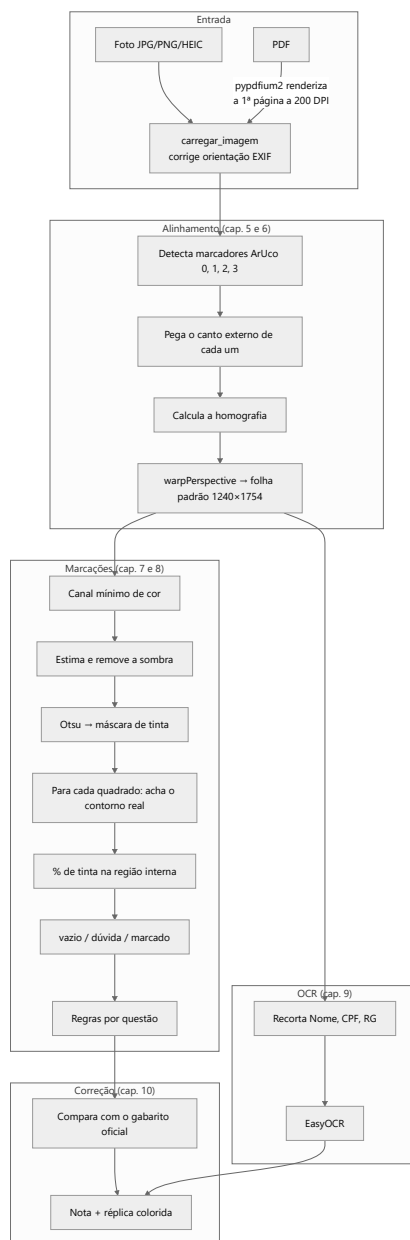
Um computador não "vê" uma folha. Ele recebe **uma matriz de números** (capítulo 3). O desafio é responder, só com contas nesses números, perguntas como "o quadrado B da questão 3 está pintado?".

A pergunta fica fácil se soubermos **exatamente em que pixels** está cada quadrado. Numa foto de celular isso muda toda vez: a folha aparece girada, inclinada, maior ou menor, com sombra. Então a estratégia é:

1. **Colocar referências conhecidas na folha** (os marcadores ArUco nos cantos).
2. **Achar essas referências na foto** e, a partir delas, **transformar a foto numa folha "padrão"**, reta e sempre do mesmo tamanho (homografia).
3. Na folha padrão, cada quadrado **está sempre no mesmo lugar**, e basta medir quanto dele está pintado.

É como encaixar um molde transparente em cima da folha: primeiro alinhamos o molde pelos cantos, depois olhamos cada janelinha.

2.3 O caminho completo



A mesma sequência, com as imagens reais que o programa produz:

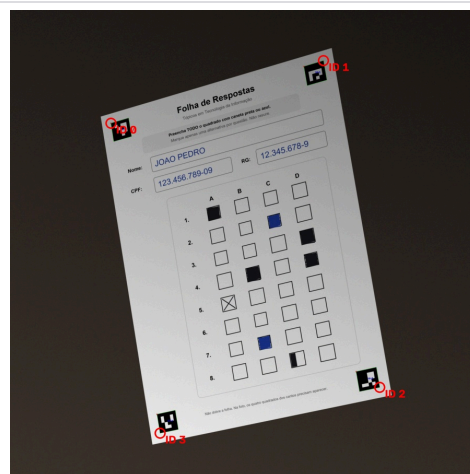
Etapa

Imagem

Foto recebida



Marcadores encontrados (círculo vermelho = canto externo usado)



Folha alinhada

Folha de Respostas
Tópicos em Tecnologia da Informação

Preencha **TODO** o quadrado com caneta preta ou azul.
Marque apenas uma alternativa por questão. Não rasure.

Nome: **JOAO PEDRO**

CPF: **123.456.789-09** RG: **12.345.678-9**

	A	B	C	D
1.	☒	☐	☐	☐
2.	☐	☐	☒	☐
3.	☐	☐	☐	☒
4.	☐	☒	☐	☒
5.	☒	☐	☐	☐
6.	☐	☐	☐	☐
7.	☐	☒	☐	☐
8.	☐	☐	☒	☐

Não dobre a folha. Na foto, os quatro quadrados dos cantos precisam aparecer.

Etapa

Máscara de tinta (branco = tinta)

Imagem

Grade detectada (verde = marcado, laranja = dúvida, cinza = vazio)

2.4 Mapa do código

Cada arquivo tem **uma responsabilidade**. Isso facilita testar e explicar cada parte separadamente.

```

trabalhoGabarito/
├── app.py                ← interface (Streamlit)
├── estilo.css            ← visual da interface
├── gabarito/
│   ├── layout.py        ← o "motor": não depende da interface
│   ├── folha.py         ← ONDE fica cada coisa na folha (coordenadas)
│   ├── alinhamento.py  ← DESENHA a folha (PNG/PDF)
│   ├── marcacoes.py     ← abre o arquivo, acha ArUco, faz a homografia
│   ├── ocr.py           ← máscara de tinta, % de preenchimento, regras
│   ├── leitura.py       ← lê Nome/CPF/RG
│   ├── correcao.py      ← ler_folha(): junta alinhamento + marcações + OCR
│   ├── visualizacao.py  ← gabarito oficial e nota
│   ├── interface_html.py ← imagem "grade detectada"
│   ├── modelos.py       ← HTML do resultado (placar, réplica da folha)
│   └── sintetico.py     ← estruturas de dados (Questao, Resultado...)
├── scripts/             ← gera folhas e fotos falsas para testes
├── tests/               ← ferramentas de linha de comando
├── amostras/            ← testes automatizados (pytest)
└── docs/               ← imagens de exemplo
                        ← você está aqui

```

A função mais importante é `ler_folha` (`gabarito/leitura.py`), que resume o pipeline em poucas linhas:

```

def ler_folha(imagem_bgr, ocr=None):
    alinhada = alinhar(imagem_bgr)                # cap. 5 e 6
    questoes = tuple(ler_marcacoes(alinhada))      # cap. 7 e 8
    identificacao, erro_ocr = None, None
    if ocr is not None:
        try:
            identificacao = ocr(alinhada)          # cap. 9
        except Exception as erro:
            erro_ocr = f"Não foi possível ler Nome/CPF/RG: {erro}"
    return LeituraFolha(questoes, alinhada, identificacao, erro_ocr)

```

Repare que um erro no OCR **não impede a correção**: a nota é o objetivo principal, e o nome é um extra.

2.5 Glossário rápido

Termo	Significado
Pixel	Menor "quadrado" de uma imagem; guarda números de cor
Folha padrão	A folha depois de endireitada: sempre 1240 × 1754 pixels
ArUco	Marcador quadrado preto e branco que o computador reconhece e identifica por número
Homografia	Transformação que "desentorta" uma imagem de um plano visto em perspectiva
Máscara	Imagem só com 0 e 255 indicando onde há algo (aqui: tinta)
Limiarização	Transformar tons de cinza em preto e branco usando um valor de corte (limiar)
Otsu	Método que escolhe o limiar automaticamente
Morfologia	Operações que alteram formas na imagem (dilatação, erosão)
Contorno	Sequência de pixels que forma a borda de uma região
OCR	Reconhecimento óptico de caracteres: transformar imagem de texto em texto
TDD	Desenvolvimento guiado por testes: escreve o teste antes do código

Imagem digital

Antes de qualquer técnica, é preciso entender **o que o computador recebe** quando abrimos uma foto.

Teoria

3.1 Uma imagem é uma matriz de números

Uma imagem digital é uma **grade de pixels**. Em uma imagem em **tons de cinza**, cada pixel guarda um **número** que diz o quão claro ele é. No formato mais comum (8 bits, `uint8`), esse número vai de **0 (preto)** a **255 (branco)**.

Um pedaço de 5×5 pixels de uma folha com um traço de caneta poderia ser:

	x=0	x=1	x=2	x=3	x=4	
y=0	[238	240	236	239	241]	← papel (claro)
y=1	[237	62	58	235	240]	
y=2	[239	55	40	61	238]	← tinta (escuro)
y=3	[240	236	64	57	237]	
y=4	[241	239	238	240	236]	

Tudo o que o software faz são **contas com essas matrizes**: comparar valores, somar, multiplicar, procurar padrões.

3.2 Coordenadas

- A origem $(0, 0)$ fica no **canto superior esquerdo**.
- **x** cresce para a **direita** (colunas); **y** cresce para **baixo** (linhas).
- Em Python/NumPy o acesso é `imagem[y, x]`, **linha primeiro**. É uma fonte clássica de confusão: o *tamanho* é descrito como "largura $\times$ altura", mas o `shape` da matriz vem como `(altura, largura)`.

No MATLAB das aulas é igual `(img(linha, coluna))`, só que os índices começam em **1** em vez de 0.

3.3 Imagens coloridas: canais

Uma imagem colorida tem **3 matrizes empilhadas**, uma por **canal**: vermelho (R), verde (G) e azul (B). Cada pixel vira um trio de números. Exemplos:

Cor	R	G	B
Branco do papel	235	235	235
Preto da caneta	25	25	30

Cor	R	G	B
Azul da caneta	30	60	160
Vermelho	200	30	40

O `shape` de uma foto colorida é `(altura, largura, 3)`.

⚠ **O OpenCV guarda as cores na ordem B, G, R**, e não R, G, B. É uma herança histórica da biblioteca. Por isso o código tem conversões como `cv2.cvtColor(img, cv2.COLOR_RGB2BGR)` sempre que passa uma imagem do Pillow (RGB) para o OpenCV (BGR).

3.4 De colorido para cinza

A conversão mais comum usa uma **média ponderada**, porque o olho humano é mais sensível ao verde:

$$\text{cinza} = 0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B$$

É o que o `rgb2gray` do MATLAB e o `cv2.COLOR_BGR2GRAY` fazem. No [capítulo 7](#) veremos que, **para achar tinta**, usamos outra ideia: o **menor** dos três canais.

3.5 Resolução e DPI

DPI (*dots per inch*, pontos por polegada) liga pixels a tamanho físico. Uma polegada tem 25,4 mm.

A folha **A4** mede 210 × 297 mm, ou seja, 8,27 × 11,69 polegadas:

DPI	Tamanho da A4 em pixels	Onde usamos
150	1240 × 1754	"Folha padrão": onde toda a leitura acontece
200	1654 × 2339	Conversão de PDF para imagem
300	2480 × 3508	Folha gerada para impressão

Com isso dá para converter qualquer medida. Um quadradinho de resposta tem 72 px na folha padrão: $72 \div 150 = 0,48$ polegada $\approx$ **12,2 mm** no papel.

3.6 Formatos de arquivo

Formato	Característica	Impacto para nós
JPG	Compressão com perda : descarta detalhes e cria pequenos "ruídos" nas bordas	Foto de celular vem assim; o algoritmo precisa tolerar ruído
PNG	Compressão sem perda	Prints e imagens geradas
HEIC	Formato padrão do iPhone	Aberto com <code>pillow-heif</code>
PDF	Documento com páginas, não uma imagem	Precisa ser renderizado (desenhado em pixels) antes

3.7 Orientação EXIF

O celular nem sempre "gira" os pixels quando você tira a foto deitado. Muitas vezes ele salva os pixels como o sensor capturou e grava uma **etiqueta EXIF** dizendo "exiba girado 90°". A galeria respeita a etiqueta, mas uma leitura ingênua dos pixels não, e a foto chega deitada.

Como aplicamos

Tudo isso está em `carregar_imagem` (`gabarito/alinhamento.py`):

```
def carregar_imagem(origem):
    """Abre JPG/PNG/HEIC (respeitando a orientação EXIF) ou a 1ª página de um PDF. Retorna
    BGR."""
    if _eh_pdf(origem):
        return cv2.cvtColor(np.array(_renderizar_pdf(origem)), cv2.COLOR_RGB2BGR)
    ...
    with Image.open(fonte) as img:
        img = ImageOps.exif_transpose(img).convert("RGB") # aplica a etiqueta EXIF
        return cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR) # Pillow (RGB) → OpenCV (BGR)
```

- **Detectar PDF:** todo arquivo PDF começa com os bytes `%PDF-`. O código olha esse "número mágico" em vez de confiar no nome do arquivo.
- **Renderizar PDF:** o PDF mede em **pontos** (1 ponto = 1/72 polegada). Para obter 200 DPI, a página é desenhada com escala `200 / 72 ≈ 2,78`: `python documento[0].render(scale=DPI_PDF / 72).to_pil()`. Usamos 200 DPI porque fica acima da folha padrão (150 DPI), sem perder detalhe, e não pesa demais.
- **Qualquer formato vira a mesma coisa:** uma matriz NumPy BGR (`altura, largura, 3`). Daqui para frente o resto do programa não precisa saber se veio de foto, print ou PDF.

Para fixar

1. Uma foto tem `shape = (3000, 4000, 3)`. Qual a largura? (Resposta: 4000 pixels; o shape é altura, largura, canais.)
2. Por que a caneta azul (`R=30, G=60, B=160`) fica escura no canal R? (Porque o valor de R é baixo: 30.)
3. Quantos milímetros tem o marcador ArUco de 100 px da folha padrão? ($100 \div 150 \times 25,4 \approx 16,9$ mm.)

A folha de respostas

Teoria: controlar a entrada simplifica o problema

Em visão computacional, **quanto mais controle temos sobre o que vai ser fotografado, mais simples e confiável fica o algoritmo**. As folhas do ENEM têm marcas de alinhamento pretas nas bordas pelo mesmo motivo.

O professor deixou o layout livre. Com isso, em vez de tentar adivinhar onde estão os quadradinhos em qualquer folha, **desenhamos a nossa**, com três vantagens:

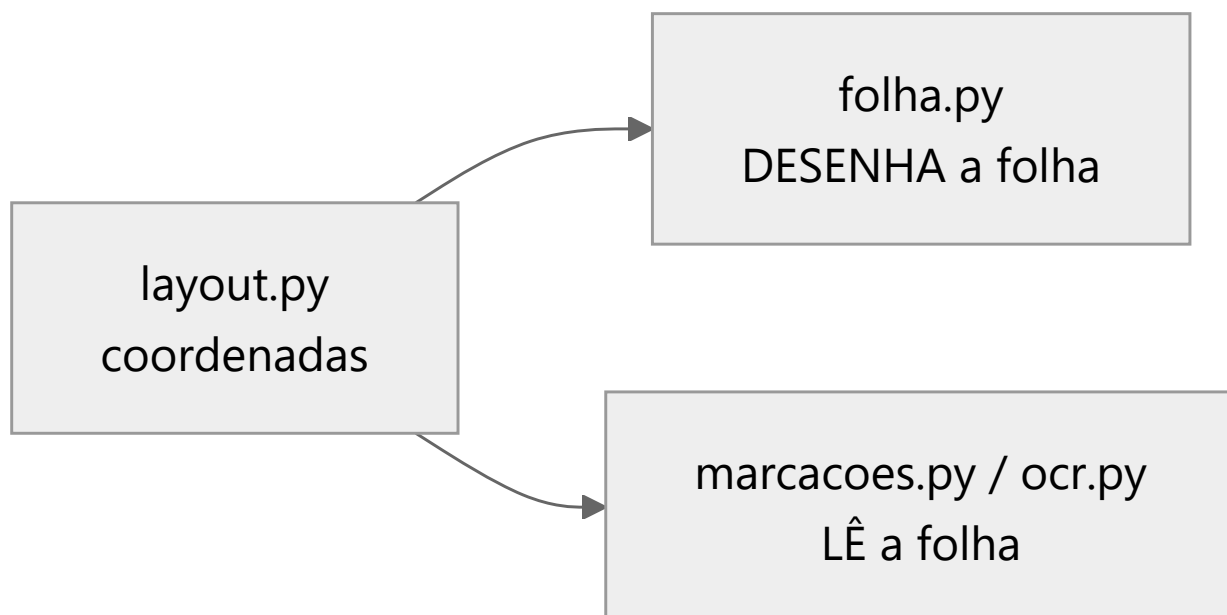
1. **Marcadores de referência** nos cantos, fáceis de achar em qualquer foto (capítulo 5).
2. **Posições conhecidas**: o programa sabe, ao pixel, onde está cada quadrado.
3. **Instruções impressas** que orientam o aluno a preencher do jeito que o algoritmo lê melhor.

O diagrama mostra uma folha de respostas com o título "Folha de Respostas" e o subtítulo "Tópicos em Tecnologia da Informação". No topo, há uma instrução: "Preencha TODO o quadrado com caneta preta ou azul. Marque apenas uma alternativa por questão. Não rasure." Abaixo, há campos para "Nome:", "CPF:" e "RG:". A grade de respostas é organizada em 8 linhas numeradas (1 a 8) e 4 colunas rotuladas A, B, C e D. Cada célula da grade contém um quadrado para ser preenchido. No rodapé, há uma nota: "Não dobre a folha. Na foto, os quatro quadrados dos cantos precisam aparecer." Os cantos da folha são marcados com quadrados pretos contendo um padrão de pontos brancos.

Como aplicamos

4.1 Uma única fonte da verdade: `layout.py`

O arquivo `gabarito/layout.py` define **todas as coordenadas** da folha, no espaço da folha padrão (1240 × 1754 px, 150 DPI). Ele é usado por **dois lados**:



Se alguém mudar o tamanho de um quadrado, o desenho e a leitura mudam juntos, e as coordenadas nunca ficam "desencontradas". Isso evita uma classe inteira de erros.

Exemplo: a posição de qualquer quadrado é calculada, não copiada à mão:

```

CELULA_LADO = 72          # lado do quadrado (≈ 12,2 mm)
GRADE_X0, GRADE_Y0 = 480, 620 # canto superior esquerdo do quadrado 1-A
PASSO_X, PASSO_Y = 150, 118  # distância entre colunas e entre linhas

def celula(questao, alternativa):
    linha = questao - 1
    coluna = ALTERNATIVAS.index(alternativa) # A=0, B=1, C=2, D=3
    return Rect(GRADE_X0 + coluna * PASSO_X, GRADE_Y0 + linha * PASSO_Y, CELULA_LADO,
                CELULA_LADO)
  
```

Por exemplo, `celula(3, "C") = Rect(x = 480 + 2·150, y = 620 + 2·118, 72, 72) = Rect(780, 856, 72, 72)`.

4.2 Os elementos da folha

Elemento	Posição / tamanho (folha padrão)	Por quê
4 marcadores ArUco	100 × 100 px (≈17 mm), a 50 px (≈8,5 mm) da borda	Grandes o bastante para aparecer em foto de longe; longe da borda para não serem cortados pela impressora
Instruções	Caixa cinza no topo	"Preencha TODO o quadrado com caneta preta ou azul. Marque apenas uma alternativa por questão. Não rasure."
Nome, CPF, RG	Caixas de 80 px de altura com borda cinza	Espaço para letra de mão; na leitura, o recorte descarta 8 px de cada lado para a borda não entrar no OCR
Grade 8 × 4	Quadrados de 72 px, bordas pretas	Mantém o formato da folha do professor
Rodapé	"Na foto, os quatro quadrados dos cantos precisam aparecer"	Orienta quem fotografa

4.3 Regras de projeto verificadas por testes

Algumas decisões do layout são **restrições** que, se quebradas, estragam a leitura. Por isso elas viraram testes automáticos em `tests/test_layout.py`:

1. **As janelas de busca não se sobrepõem.** Na leitura, cada quadrado é procurado numa janela 30% maior que ele (capítulo 8). Com 72 px de lado, a janela tem $72 + 2 \cdot 22 = 116$ px. Por isso o passo entre linhas é 118 px e não menos: se as janelas se sobrepusessem, o programa poderia "achar" o quadrado da questão vizinha.
2. **Nada encosta nos marcadores.** O ArUco precisa de uma **zona de silêncio** branca em volta para ser detectado.

Um caso real que o teste pegou: na primeira versão, a moldura cinza da grade e o texto do rodapé encostavam no marcador inferior direito, e ele **deixou de ser detectado**. A falha apareceu no teste da folha; criamos o teste da "zona de silêncio" e reorganizamos o layout. É um bom exemplo de por que testar o layout vale a pena.

4.4 Desenhando a folha: `folha.py`

A folha é desenhada com a biblioteca **Pillow** (formas e texto) e o **OpenCV** (marcadores):

```
marca = cv2.aruco.generateImageMarker(dicionario, id_marca, r.w * s) # gera o desenho do
ArUco
img.paste(Image.fromarray(marca).convert("RGB"), (r.x * s, r.y * s)) # cola na posição do
layout
...
d.rectangle(_escalar(layout.celula(questao, alternativa), s), outline=PRETO, width=3 * s) #
quadrado
```

- `escala = 1` desenha na folha padrão (150 DPI), usada nos testes.
- `escala = 2` desenha a 300 DPI para impressão; o PDF é salvo com `resolution=300`, e assim a página sai exatamente em tamanho A4.

Marcadores ArUco: achando os cantos da folha

Teoria

5.1 Marcadores fiduciais

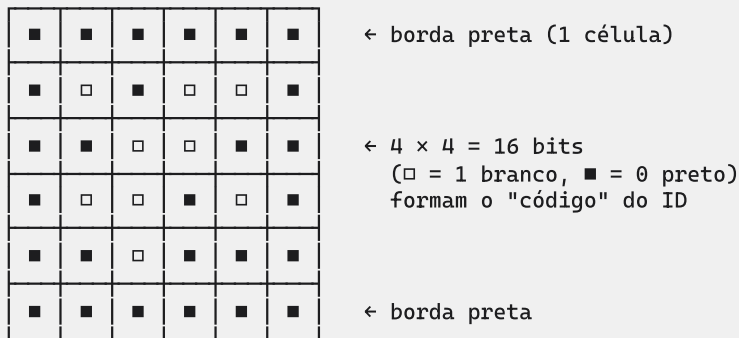
Um **marcador fiducial** é um desenho feito para ser **fácil de encontrar e identificar** por um computador. O QR Code é o exemplo mais conhecido. Em visão computacional e realidade aumentada usam-se muito o **ArUco** e o **AprilTag**, porque são simples, rápidos e dizem **exatamente onde estão seus 4 cantos**.

Precisamos de duas informações sobre cada canto da folha: - **onde** ele está na foto (coordenadas em pixels); - **qual** canto ele é (superior esquerdo? inferior direito?).

O ArUco resolve as duas.

5.2 Como um ArUco é construído

Usamos o dicionário **DICT_4X4_50** : **4 × 4 bits** internos e **50 marcadores diferentes** (IDs 0 a 49).

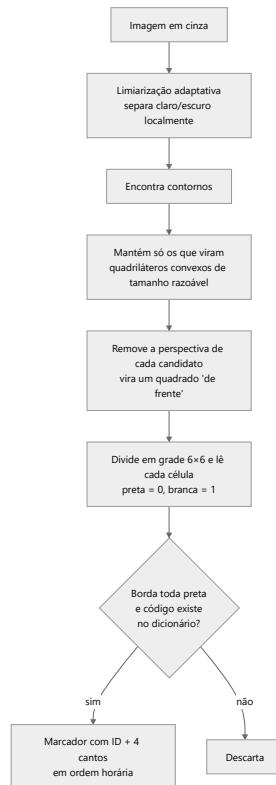


(padrão ilustrativo)

- A **borda preta** torna o quadrado fácil de achar.
- Os **16 bits de dentro** formam o código. Os padrões do dicionário foram escolhidos para serem **bem diferentes entre si, inclusive quando girados**. A diferença é medida pela **distância de Hamming** (quantos bits mudam). Isso traz duas consequências:
 - se alguns bits forem lidos errado (sombra, desfoque), o detector ainda identifica o marcador correto ou descarta a leitura, em vez de confundir IDs;
 - como o padrão **não é simétrico**, o detector descobre a **rotação**: sabe qual canto é o "superior esquerdo do marcador", mesmo com a foto de cabeça para baixo.

5.3 Como o detector encontra os marcadores

O `cv2.aruco.ArucoDetector` faz, resumidamente:

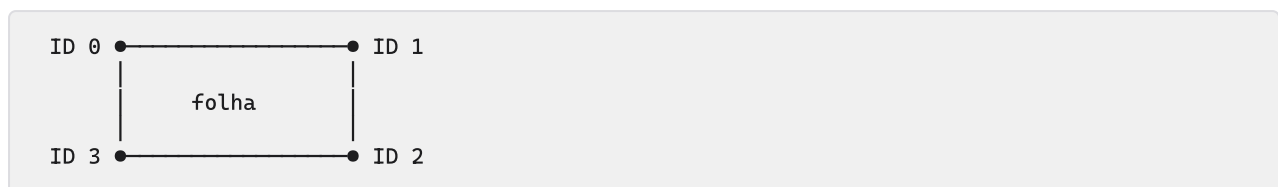


A saída é, para cada marcador, o **ID** e os **4 cantos** em **ordem horária**, começando pelo canto superior esquerdo do próprio marcador. É essa ordem que nos deixa descobrir a orientação da folha.

Como aplicamos

5.4 Um ID para cada canto

Na folha, cada canto recebe um marcador com ID fixo:

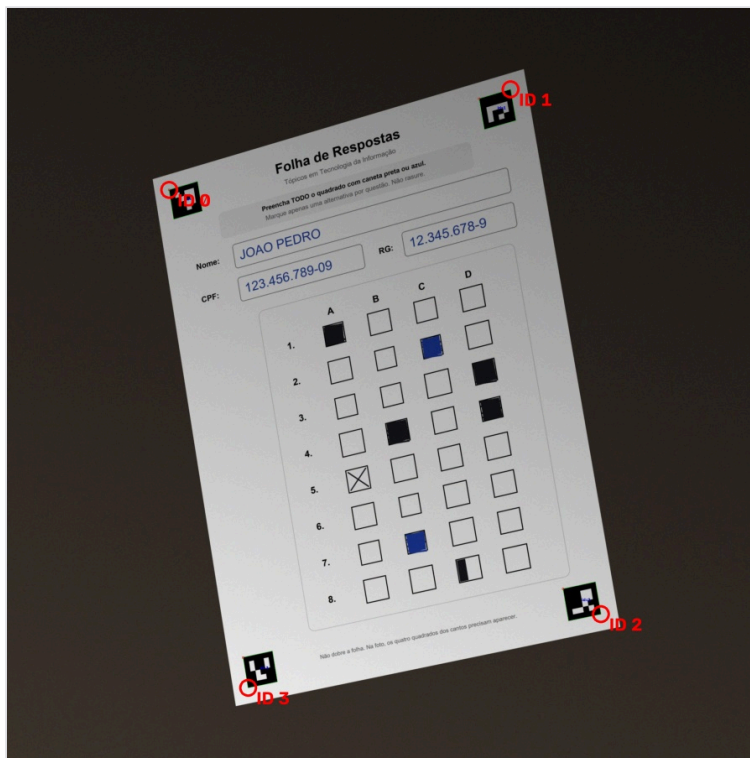


(● = canto **externo** do marcador, o mais próximo da quina do papel)

Detalhe que simplifica muito o código: os cantos do ArUco vêm em ordem horária `[0]=sup-esq`, `[1]=sup-dir`, `[2]=inf-dir`, `[3]=inf-esq` do próprio marcador. Com os marcadores impressos "em pé", o **canto externo do marcador de ID i** é justamente o **canto de índice i**:

```
# canto externo do marcador i é o seu canto de índice i (ver layout.py)
encontrados[id_marcador] = c.reshape(4, 2)[id_marcador] / fator
```

Como o índice é **do marcador**, e não da foto, isso continua valendo com a folha girada ou de cabeça para baixo.



Contorno verde: marcador detectado. Círculo vermelho: canto externo usado na homografia.

5.5 Detalhes de robustez

Detalhe	Código	Por quê
Reduzir fotos grandes	Lado maior ≤ 2000 px na detecção; as coordenadas são multiplicadas de volta	Fotos de celular têm 4000 px ou mais; detectar em tamanho menor é mais rápido e, em geral, mais estável
Segunda tentativa	Se faltar marcador, detecta de novo na resolução original	Marcadores muito pequenos na foto podem sumir na redução
Refinamento subpixel	<code>CORNER_REFINE_SUBPIX</code>	Posição do canto com precisão melhor que 1 pixel, e homografia mais exata
Mensagem útil	<code>MarcadoresNaoEncontrados(faltando=[2, 3])</code> → "Não encontrei os cantos inferior direito e inferior esquerdo"	O usuário sabe o que refazer

5.6 Por que não algo mais simples?

Consideramos três abordagens antes de programar:

Abordagem	Vantagem	Por que não escolhemos
A) ArUco 	Identifica qual canto é qual; robusto a rotação e iluminação	—
B) Quadrados pretos simples	Visualmente mais simples	Todos os quadrados são iguais: a orientação precisa ser "adivinhada" pela posição, o que falha com a folha muito girada ou de cabeça para baixo. Sombras e textos pretos geram falsos positivos
C) Sem marcadores (achar a borda do papel e a grade)	Funciona com a folha original do professor	Muito sensível a fundo claro, sombra, folha dobrada; muito mais código de detecção

Limitação assumida: se um marcador sair cortado ou coberto na foto, a folha não é lida, e o app pede outra foto. Por isso a **folha original do professor** (sem marcadores) não é aceita.

Homografia: endireitando a folha

Teoria

6.1 O problema da perspectiva

Quando fotografamos uma folha "de lado", ela não aparece como um retângulo: vira um **quadrilátero qualquer**. O lado mais distante da câmera fica menor, e linhas paralelas deixam de parecer paralelas. Girar e redimensionar não basta para desfazer isso.

6.2 A família das transformações geométricas

Cada tipo de transformação consegue "desfazer" certos efeitos. Os **graus de liberdade** são quantos números precisamos para descrevê-la:

Transformação	O que faz	Graus de liberdade	Preserva
Translação	Desloca	2	Tudo, menos a posição
Rígida (Euclidiana)	Desloca + gira	3	Distâncias e ângulos
Similaridade	+ muda escala	4	Ângulos e proporções
Afim	+ estica e cisalha	6	Retas paralelas continuam paralelas
Projetiva (homografia)	+ perspectiva	8	Só retas continuam retas

A foto de uma folha em perspectiva só é corrigida pela **homografia**, a mais geral das transformações que levam retas em retas.

Ideia-chave: **a folha é um plano**. Toda foto de um plano, de qualquer ângulo, se relaciona com a folha "vista de frente" por **uma única homografia**. Isso vale para o plano inteiro, não só para os cantos.

6.3 A matemática (sem medo)

A homografia é uma **matriz 3×3** :

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}$$

Para levar um ponto (x, y) da foto para (x', y') na folha:

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \text{e depois} \quad x' = u / w \quad y' = v / w$$

Esse "1" extra e a divisão por `w` são as **coordenadas homogêneas**. A divisão por `w` é o que cria o efeito de perspectiva: pontos "mais longe" são divididos por um número maior e ficam mais próximos entre si.

- `h13` e `h23` : **translação**.
- `h11`, `h12`, `h21`, `h22` : **rotação, escala, cisalhamento** (a parte afim).
- `h31` e `h32` : **perspectiva**. Se forem 0, então `w = 1` e a transformação é só afim.

6.4 Quantos pontos são necessários?

A matriz tem **8 incógnitas** (o último elemento é fixado em 1). Cada par de pontos correspondentes $(x, y) \rightarrow (x', y')$ dá **2 equações** (uma para x' , outra para y'). Logo, **4 pontos dão 8 equações** e determinam a homografia exatamente.

É por isso que usamos **4 marcadores**: 3 cantos não bastariam para a perspectiva.

`cv2.getPerspectiveTransform` monta e resolve esse sistema linear 8×8 .

6.5 Aplicando na imagem: mapeamento inverso e interpolação

Um jeito ingênuo de endireitar seria percorrer cada pixel da foto e "jogá-lo" na folha. Isso deixa **buracos**, porque nem todo pixel de destino recebe alguém. O `cv2.warpPerspective` faz o contrário (**mapeamento inverso**):

Para **cada pixel da folha de destino**, calcula de **onde ele vem** na foto (usando H^{-1}) e busca a cor lá.

O ponto de origem quase nunca cai exatamente num pixel inteiro, por exemplo $(812, 25; 403, 5)$. É preciso **interpolar**:

Método	Como funciona	Resultado
Vizinho mais próximo	Pega o pixel mais perto	Rápido, mas serrilhado
Bilinear	Média ponderada dos 4 vizinhos	Suave, o padrão para rotações
Área (<code>INTER_AREA</code>)	Média de todos os pixels que "caem" dentro do pixel de destino	O melhor para reduzir imagens

Exemplo de interpolação bilinear. Os 4 vizinhos valem:

$$\begin{array}{ll} (0,0) = 100 & (1,0) = 200 \\ (0,1) = 50 & (1,1) = 150 \end{array}$$

Queremos o valor em $(0, 25; 0, 5)$: 1. Na horizontal, linha de cima: $100 + 0,25 \cdot (200 - 100) = 125$ 2. Na horizontal, linha de baixo: $50 + 0,25 \cdot (150 - 50) = 75$ 3. Na vertical, entre as duas: $125 + 0,5 \cdot (75 - 125) = 100$

Como aplicamos

6.6 O código

```
origem = np.float32([encontrados[i] for i in range(4)]) # cantos na foto
destino = np.float32([layout.CANTOS_EXTERNOS[i] for i in range(4)]) * SUPERAMOSTRAGEM # cantos na folha padrão
homografia = cv2.getPerspectiveTransform(origem, destino)
grande = cv2.warpPerspective(imagem_bgr, homografia,
                             (layout.LARGURA * 2, layout.ALTURA * 2),
                             flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE)
return cv2.resize(grande, (layout.LARGURA, layout.ALTURA), interpolation=cv2.INTER_AREA)
```

Os cantos de destino são as posições dos marcadores no layout: (50, 50), (1190, 50), (1190, 1704) e (50, 1704). Assim, depois da transformação, **todo o resto da folha cai exatamente nas coordenadas do layout.py**.

6.7 Um exemplo com números reais

Na foto de exemplo (img/02_foto.jpg, 2716 × 2716 px) os cantos externos foram encontrados em:

Marcador	Na foto (x, y)	Na folha padrão
ID 0	(583, 654)	(50, 50)
ID 1	(1811, 292)	(1190, 50)
ID 2	(2136, 2180)	(1190, 1704)
ID 3	(867, 2445)	(50, 1704)

A homografia calculada foi:

$$H = \begin{bmatrix} 0,9685 & -0,1530 & -413,02 \\ 0,2755 & 0,9264 & -714,97 \\ 4,9 \cdot 10^{-5} & 3,1 \cdot 10^{-6} & 1 \end{bmatrix}$$

Leitura da matriz: o bloco 0,9685 / 0,2755 / -0,153 / 0,9264 é quase uma rotação. A primeira coluna aponta ~16° e a segunda ~9°; se fosse rotação pura os dois ângulos seriam iguais, e a diferença vem da perspectiva e do cisalhamento (a foto foi girada 12° e inclinada). -413 e -715 deslocam a folha para a origem; h31 = 0,000049 é pequeno, mas não zero, e é a perspectiva. Conferindo o canto ID 2:

```
u = 0,9685 · 2136 - 0,1530 · 2180 - 413,02 ≈ 1322,2
v = 0,2755 · 2136 + 0,9264 · 2180 - 714,97 ≈ 1893,1
w = 0,0000488 · 2136 + 0,0000031 · 2180 + 1 ≈ 1,111
x' = u / w ≈ 1190,1    y' = v / w ≈ 1703,9    ✓ (esperado: 1190, 1704)
```

O $w = 1,111$ (e não 1) mostra a divisão de perspectiva agindo: esse canto estava "mais longe" na foto simulada.

Folha de Respostas
Tópicos em Tecnologia da Informação

Preencha **TODOS** o quadrado com caneta preta ou azul.
Marque apenas uma alternativa por questão. Não rasure.

Nome: **JOAO PEDRO**

CPF: **123.456.789-09** RG: **12.345.678-9**

	A	B	C	D
1.	☒	☐	☐	☐
2.	☐	☐	☒	☐
3.	☐	☐	☐	☒
4.	☐	☒	☐	☒
5.	☒	☐	☐	☐
6.	☐	☐	☐	☐
7.	☐	☒	☐	☐
8.	☐	☐	☒	☐

Não dobre a folha. Na foto, os quatro quadradinhos dos cantos precisam aparecer.

6.8 Por que gerar o dobro e depois reduzir?

O `warpPerspective` **não aceita** `INTER_AREA`. Quando a folha na foto é **maior** que a folha padrão (o caso normal em fotos de celular), a transformação está **reduzindo** a imagem, e a interpolação bilinear pura pode "pular" linhas finas (o traço da borda de um quadrado tem só 3 px). A solução:

1. `warpPerspective` para **2× o tamanho** (2480×3508), com bilinear;
2. `resize` para 1240×1754 com `INTER_AREA`, que faz a média de cada bloco 2×2 .

Isso se chama **superamostragem**.

6.9 BORDER_REPLICATE

Se algum pixel de destino "vier de fora" da foto, ele recebe a cor da borda mais próxima em vez de preto. Isso evita faixas pretas artificiais, que seriam confundidas com tinta.

6.10 O que a homografia não corrige

- **Papel dobrado ou curvado:** deixa de ser um plano, e uma única homografia não descreve a foto. Pequenas curvaturas são absorvidas pelo refinamento local (capítulo 8); dobras fortes, não.
- **Distorção de lente** (efeito "olho de peixe"): celulares modernos já corrigem quase toda.

Segmentação da tinta: separando tinta de papel

Segmentar é dividir a imagem em regiões com significado. Aqui queremos a divisão mais simples possível: cada pixel é **tinta** ou **papel**. O resultado é uma **máscara**, uma imagem onde 255 (branco) = tinta e 0 (preto) = papel.

Parece fácil ("escuro é tinta"), mas três problemas aparecem na prática: 1. **Cor**: tinta azul não é tão escura quanto a preta. 2. **Sombra**: papel na sombra pode ficar mais escuro que tinta no sol. 3. **Onde cortar**: qual valor separa "escuro" de "claro"?

Teoria

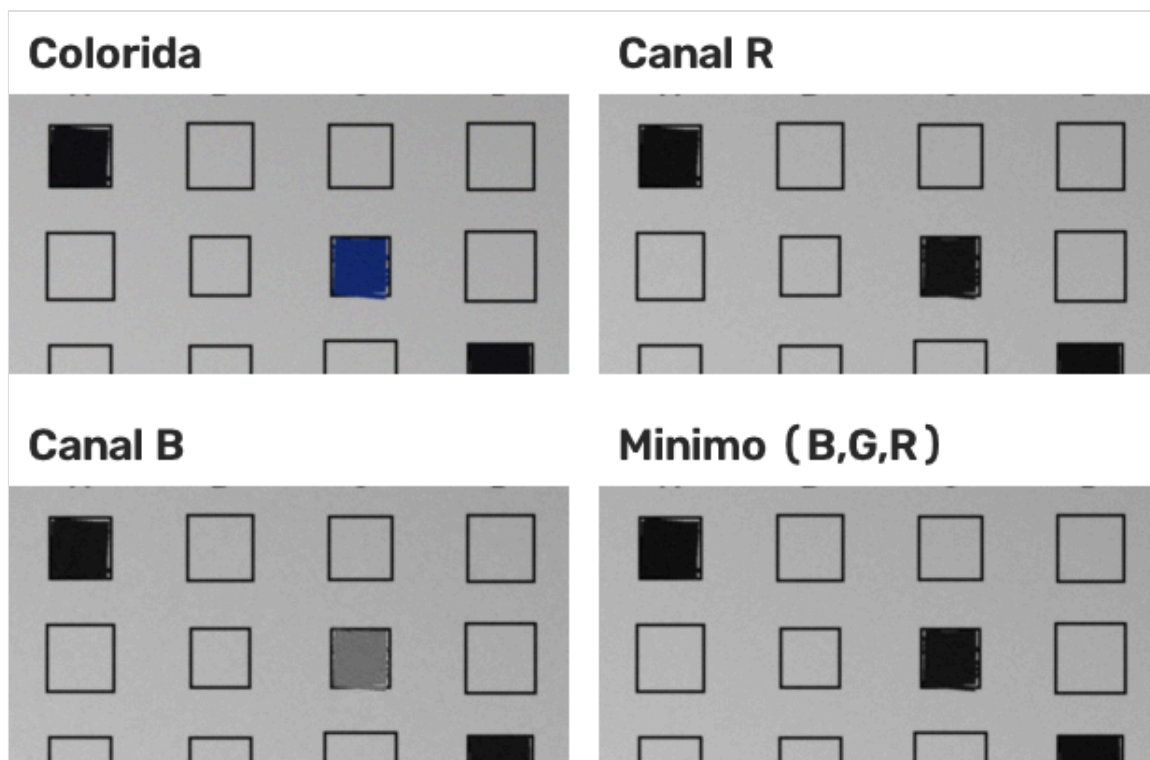
7.1 Problema 1: cor. Usar o menor canal

A conversão comum para cinza ($0,299 \cdot R + 0,587 \cdot G + 0,114 \cdot B$) dá peso **pequeno ao azul**. Resultado: tinta azul clara vira um cinza médio, perigosamente perto do papel. Uma ideia melhor para **achar tinta** é usar o **menor valor entre R, G e B**:

- **papel branco** é claro em **todos** os canais, então o mínimo continua alto;
- **qualquer tinta colorida** é escura em **pelo menos um** canal, então o mínimo fica baixo.

Cor	R, G, B	Cinza ponderado	Mínimo
Papel branco	235, 235, 235	235	235
Papel amarelado	240, 230, 200	230	200
Caneta preta	25, 25, 30	25	25
Caneta azul	30, 60, 160	62	30
Caneta azul clara	90, 140, 230	135	90
Caneta vermelha	200, 30, 40	82	30

A caneta azul clara passa de 135 (ambíguo) para 90 (claramente tinta).



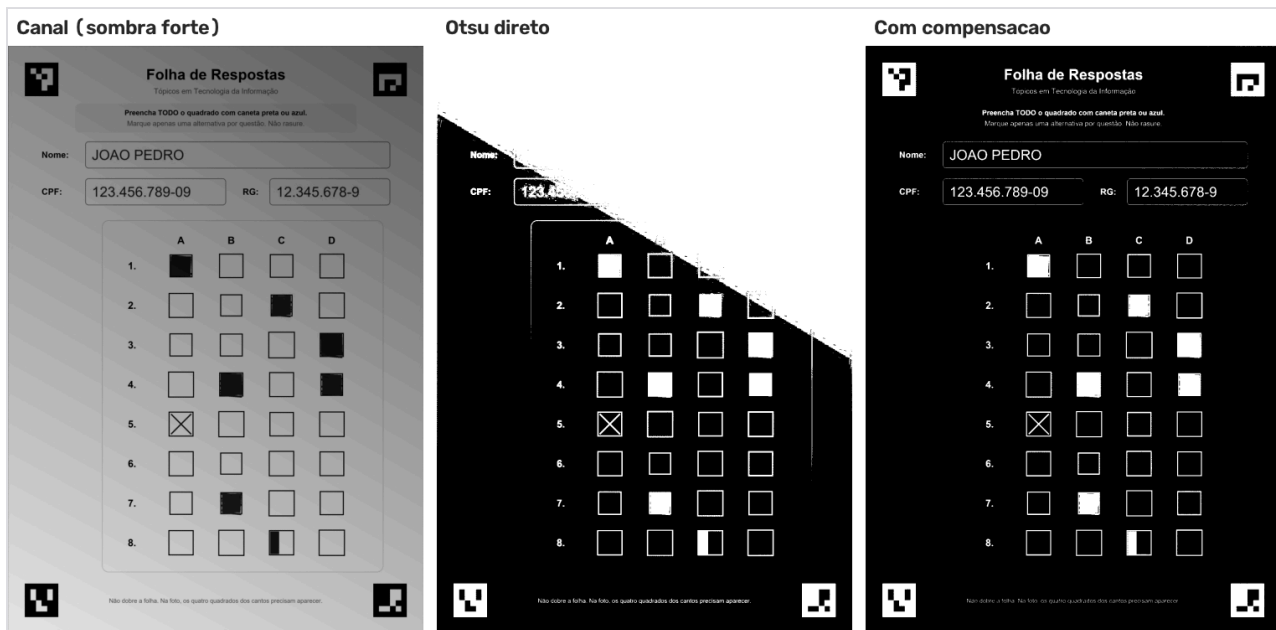
Na imagem colorida há uma marcação preta e uma azul. No **canal B** a azul fica clara, quase some. No **canal R** ela fica escura. O **mínimo** fica escuro para as duas.

7.2 Problema 2: iluminação desigual

Limiarização global usa um único valor de corte para a imagem inteira: `pixel < limiar → tinta`. Com sombra, não existe um valor que funcione em todo lugar:

	lado iluminado	lado na sombra
papel	240	120
tinta	80	40

Um limiar de 100 marca como tinta o papel da sombra? Não ($120 > 100$). Um limiar de 130? Marca ($120 < 130$). E um limiar de 70 perde a tinta do lado iluminado ($80 > 70$). **Não há limiar global que separe as duas colunas.** A figura mostra o que acontece numa foto com sombra forte:



Esquerda: canal com sombra forte. Meio: **Otsu direto**, e 37% da folha vira "tinta" (o triângulo branco é a região na sombra). Direita: **com compensação**, só 5%, que é a tinta de verdade.

7.3 A solução: estimar o fundo e dividir

Se soubermos **como seria o papel sem tinta** em cada ponto (o "fundo"), basta **dividir**:

$$\text{normalizada} = \text{canal} / \text{fundo} \times 255$$

No exemplo da tabela acima:

	Canal	Fundo (papel ali)	Normalizada
Papel no sol	240	240	$240/240 \times 255 = 255$
Tinta no sol	80	240	$80/240 \times 255 = 85$
Papel na sombra	120	120	$120/120 \times 255 = 255$
Tinta na sombra	40	120	$40/120 \times 255 = 85$

A sombra **desaparece**: a sombra multiplica tudo por um fator, e a divisão cancela esse fator. (Em fotografia isso se chama *flat-field correction*.)

A questão passa a ser: **como estimar o fundo** se a folha está cheia de tinta? Com **morfologia**.

7.4 Morfologia: dilatação

Operações morfológicas passam uma **janela** (o *elemento estruturante*) por cima da imagem. As duas básicas, em tons de cinza:

- **Dilatação**: cada pixel recebe o **maior** valor dentro da janela. Regiões **claras crescem**.
- **Erosão**: cada pixel recebe o **menor** valor dentro da janela. Regiões **escuras crescem**.

(Nas aulas vimos as duas em imagens binárias, com `imdilate` / `imerode` ; em cinza a ideia é a mesma, com máximo e mínimo.)

Exemplo em uma linha de pixels, dilatação com janela de 3:

```
original:  240  240   60   50  240  240
dilatada:  240  240  240  240  240  240
```

O traço escuro (60, 50) tinha 2 pixels, menos que a janela, e **sumiu**. Sobrou só o papel. Mas atenção: se a mancha de tinta for **maior que a janela**, o meio dela sobrevive. Por isso **a janela precisa ser maior que a maior mancha de tinta**.

Depois da dilatação aplicamos um **desfoque gaussiano**, que suaviza a estimativa (a dilatação deixa um aspecto de "blocos").

7.5 Problema 3: onde cortar. O método de Otsu

Depois de normalizar, o histograma (quantos pixels existem de cada valor) tem **dois morros**: um de tinta (escuro) e um de papel (claro). O **método de Otsu** escolhe automaticamente o limiar que **melhor separa os dois grupos**.

Para cada limiar possível t , divide os pixels em dois grupos e calcula a **variância entre classes**:

$$\sigma^2_{\text{entre}}(t) = w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$$

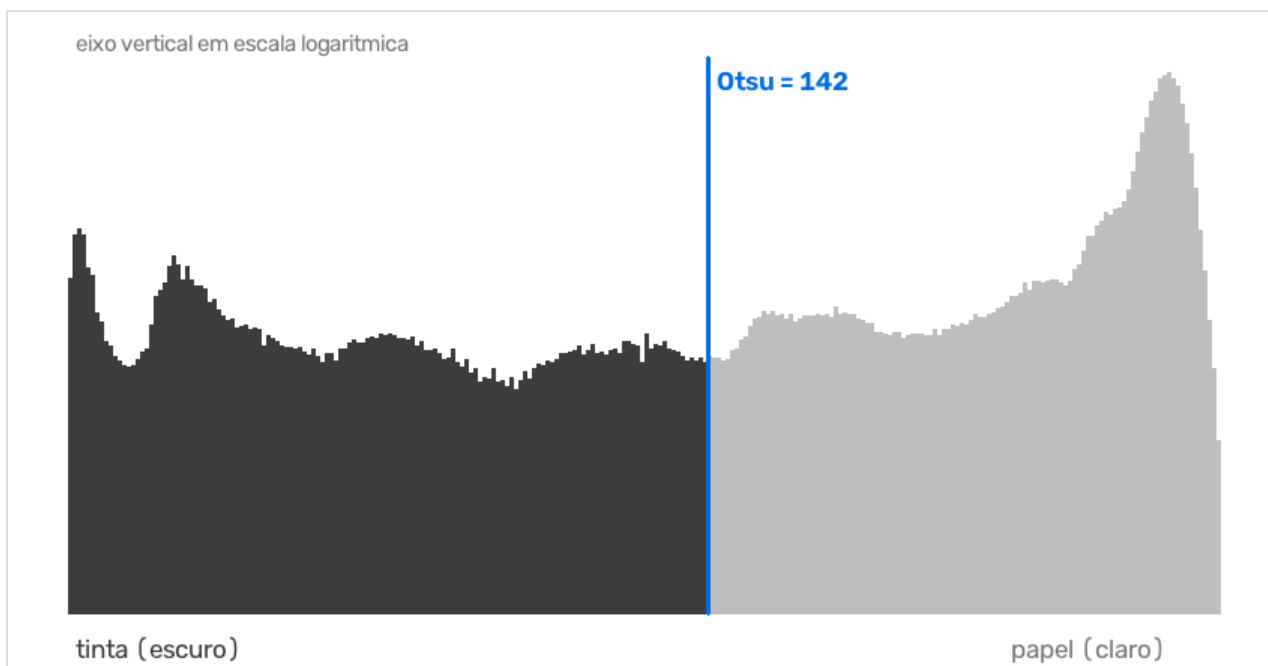
- w_0, w_1 = fração de pixels em cada grupo
- μ_0, μ_1 = média de cada grupo

O melhor t é o que **maximiza** esse valor: grupos com médias bem distantes e ambos com tamanho relevante.

Exemplo com 10 pixels: 20, 25, 30, 200, 210, 215, 220, 225, 230, 235

Corte	Grupo escuro	Grupo claro	$w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$
entre 25 e 30	20, 25 $\rightarrow \mu_0 = 22,5$	8 pixels $\rightarrow \mu_1 = 195,6$	$0,2 \cdot 0,8 \cdot 173,1^2 \approx \mathbf{4.796}$
entre 30 e 200	20, 25, 30 $\rightarrow \mu_0 = 25$	7 pixels $\rightarrow \mu_1 = 219,3$	$0,3 \cdot 0,7 \cdot 194,3^2 \approx \mathbf{7.927}$ ✓
entre 200 e 210	4 pixels $\rightarrow \mu_0 = 68,75$	6 pixels $\rightarrow \mu_1 = 222,5$	$0,4 \cdot 0,6 \cdot 153,75^2 \approx \mathbf{5.673}$

O corte entre 30 e 200 vence, que é exatamente o que um humano faria olhando os números.



Histograma da folha normalizada do exemplo. O Otsu escolheu 142. À esquerda (escuro): tinta, texto impresso, bordas. À direita: papel. O eixo vertical está em escala logarítmica, porque há muito mais papel que tinta.

Como aplicamos

Tudo acontece em `etapas_mascara` (`gabarito/marcacoes.py`):

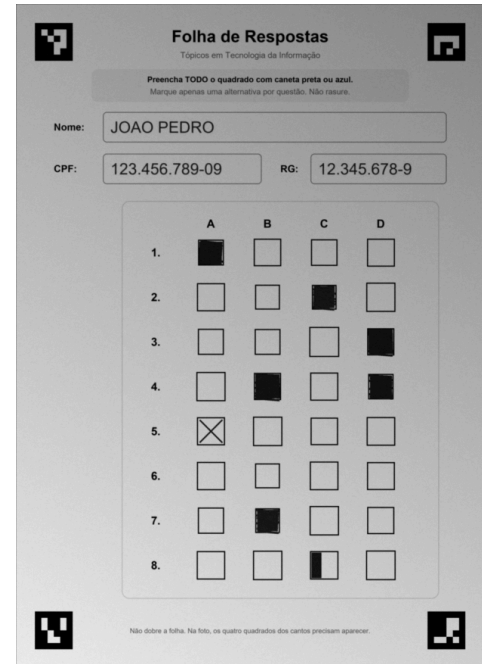
```
KERNEL_FUNDO = 101          # janela da dilatação (px)
LIMIAR_MIN, LIMIAR_MAX = 100, 200 # faixa permitida para o limiar

def etapas_mascara(folha_bgr):
    canal = folha_bgr.min(axis=2) # 7.1 menor canal
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (KERNEL_FUNDO, KERNEL_FUNDO))
    fundo = cv2.GaussianBlur(cv2.dilate(canal, kernel), (0, 0), 15) # 7.4 fundo
    normalizada = cv2.divide(canal, np.maximum(fundo, 1), scale=255) # 7.3 divisão
    limiar_otsu, _ = cv2.threshold(normalizada, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    # 7.5
    limiar = float(np.clip(limiar_otsu, LIMIAR_MIN, LIMIAR_MAX))
    mascara = np.where(normalizada < limiar, 255, 0).astype(np.uint8)
    ...
```

Etapa

Canal mínimo

Imagem



Folha de Respostas
Tópicos em Tecnologia da Informação

Preencha **TODO** o quadrado com caneta preta ou azul.
Marque apenas uma alternativa por questão. Não rasure.

Nome: JOAO PEDRO

CPF: 123.456.789-09 RG: 12.345.678-9

	A	B	C	D
1.	☒	☐	☐	☐
2.	☐	☐	☒	☐
3.	☐	☐	☐	☒
4.	☐	☒	☐	☒
5.	☒	☐	☐	☐
6.	☐	☐	☐	☐
7.	☐	☒	☐	☐
8.	☐	☐	☒	☐

Não dobre a folha. Na foto, os quatro quadrados dos cantos precisam aparecer.

Fundo estimado (dilatação 101×101 + desfoque): a tinta sumiu e sobrou a iluminação



Etapa

Normalizada (canal ÷ fundo): papel uniforme

Imagem

Máscara final (Otsu)

Por que esses números?

Valor	Justificativa
Janela de 101 × 101 px	O maior borrão de tinta esperado é um quadrado todo preenchido: 72 × 72 px. A janela precisa ser maior (seção 7.4). 101 é ímpar (tem centro) e dá folga para rabiscos que saem do quadrado
Desfoque $\sigma = 15$	Suaviza os "degraus" da dilatação sem apagar variações reais de luz, que são lentas
<code>np.maximum(fundo, 1)</code>	Evita divisão por zero

Valor	Justificativa
Limiar preso entre 100 e 200	O Otsu sempre devolve algum corte, mesmo quando a imagem quase não tem tinta. Nesse caso, ele poderia separar "papel levemente mais claro" de "papel levemente mais escuro" e transformar ruído em tinta. Prender o valor impede escolhas absurdas. Na foto de exemplo o Otsu deu 142, dentro da faixa, e foi usado sem alteração

Testes que garantem isso

Em `tests/test_marcacoes.py` : - `test_sombra_forte_ao_vira_marcacao` : escurece a folha em degradê até 45% do brilho e exige que todas as questões continuem **em branco**; -

`test_le_preenchimento_preto_azul_multiplo_e_x` : marcação **azul** precisa ser lida como marcada.

Leitura das marcações

Com a folha alinhada (capítulo 6) e a máscara de tinta pronta (capítulo 7), falta responder, para cada um dos 32 quadrados: **está marcado?** Depois aplicamos as regras de cada questão.

Teoria

8.1 Contornos

Um **contorno** é a lista de pixels da **borda** de uma região branca numa imagem binária. O OpenCV encontra contornos com o algoritmo de *border following* de **Suzuki e Abe (1985)**: percorre a imagem e, ao encontrar a borda de uma região, "caminha" por ela até voltar ao início.

Opções usadas: - `RETR_EXTERNAL` : devolve **só os contornos mais externos**. Um quadrado com um X dentro gera um contorno (o do quadrado); o X, que está dentro, é ignorado. -

`cv2.boundingRect(contorno)` : o **menor retângulo alinhado aos eixos** que contém o contorno. Com ele obtemos posição e tamanho do quadrado impresso.

8.2 Medir por proporção, não por contagem absoluta

"Quantos pixels de tinta há no quadrado?" depende do tamanho do quadrado. "**Que fração** do quadrado tem tinta?" não depende. Por isso medimos:

$$\text{preenchimento} = \text{pixels de tinta na região} \div \text{total de pixels da região}$$

Um número entre 0 (vazio) e 1 (totalmente pintado).

8.3 Decisão com zona de dúvida

Um classificador com um único limite ("acima de 50% = marcado") força uma decisão mesmo quando o caso é ambíguo: um X forte pode passar do limite e virar "resposta", e um preenchimento fraco pode sumir. Uma técnica comum é criar uma **opção de rejeição**: além de "sim" e "não", existe "**não tenho certeza**". Quando a decisão errada é cara (dar ou tirar ponto de um aluno), é melhor sinalizar do que chutar.

Como aplicamos

8.4 Passo 1: achar o quadrado real (refinamento local)

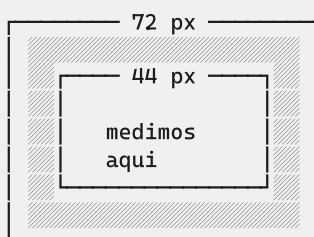
A homografia coloca a folha no lugar, mas a **impressão** pode ter pequenas variações: um quadrado um pouco deslocado ou de tamanho diferente. Então, em vez de confiar cegamente na posição do `layout.py`, procuramos o quadrado de verdade perto dela:

```
def localizar_quadrado(mascara, esperado):
    janela = esperado.expandir(layout.FOLGA_BUSCA)           # 30% maior de cada lado
    recorte = mascara[janela.y : janela.y + janela.h, janela.x : janela.x + janela.w]
    contornos, _ = cv2.findContours(recorte, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    melhor = esperado                                         # plano B: posição do template
    for contorno in contornos:
        x, y, w, h = cv2.boundingRect(contorno)
        if not (0.75 <= w / esperado.w <= 1.30 and 0.75 <= h / esperado.h <= 1.30):
            continue                                           # tamanho incompatível com um
                                                                # quadrado
        # ... fica com o candidato cujo centro está mais perto do esperado
    return melhor
```

Regra	Valor	Por quê
Janela de busca	+30% por lado (72 → 116 px)	Cobre desalinhamentos de impressão sem invadir o quadrado vizinho (garantido pelo teste de layout)
Tamanho aceito	75% a 130% do esperado	Aceita impressões com escala diferente; rejeita pedaços de texto ou rabiscos grandes
Critério de escolha	Centro mais próximo	Se houver mais de um candidato, o mais provável é o que está onde deveria
Plano B	Posição do template	Se nada plausível for encontrado (ex.: rabisco enorme borrou tudo), ainda medimos no lugar esperado

8.5 Passo 2: medir só a parte de dentro

A **borda impressa** do quadrado também é preta e apareceria na máscara como tinta. Por isso encolhemos o quadrado **20% de cada lado** e medimos só no miolo:



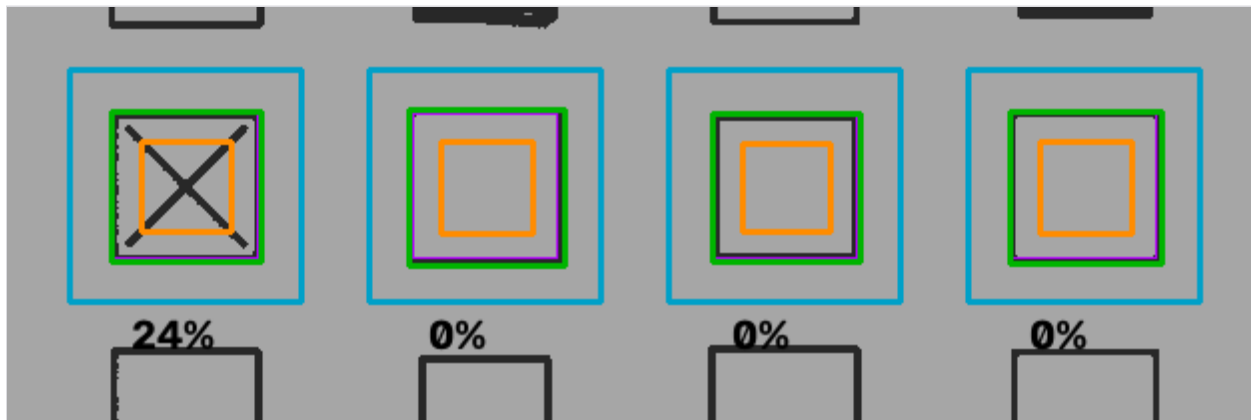
▨ = margem ignorada (14 px de cada lado)

região interna: 44 × 44 = 1936 pixels

```
def medir_preenchimento(mascara, quadrado):
    interno = quadrado.encolher(0.20)
    regioao = mascara[interno.y : interno.y + interno.h, interno.x : interno.x + interno.w]
    return np.count_nonzero(regiao) / regioao.size
```

Exemplo: se 1.000 dos 1.936 pixels internos forem tinta, o preenchimento é $1000 \div 1936 \approx 52\%$.

Por que isso torna o sistema tolerante a quadrados de tamanhos diferentes? Mesmo que o refinamento falhe e usemos a posição do template, a região interna (44 px) fica **dentro** de quadrados impressos entre 85% e 115% do tamanho: - quadrado a 85%: lado ≈ 61 px, borda a ± 30 px do centro; a região interna vai só até ± 22 px ✓ - quadrado a 115%: lado ≈ 83 px, com ainda mais folga ✓



Questão 5 da foto de exemplo. **Azul:** janela de busca. **Verde:** quadrado encontrado. **Laranja:** região interna medida. Na alternativa A (X desenhado), 24% da região interna é tinta; nas outras, 0%.

8.6 Passo 3: classificar cada quadrado

```
LIMITE_VAZIO = 0.15
LIMITE_MARCADO = 0.50
```

Preenchimento	Estado	Exemplos reais medidos
$\leq 15\%$	vazio	quadrados em branco: 0%
entre 15% e 50%	dúvida	X: 24% · só um terço pintado: 33%
$\geq 50\%$	marcado	quadrado todo pintado (preto ou azul): 100%

Por que esses valores? - **15%**: bem acima de um quadrado vazio (0% nas fotos de teste, porque a borda impressa fica fora da região interna), mas abaixo de um X feito com caneta de espessura normal (24% no teste). Com fotos reais, vale conferir esses números no `avaliar.py`. - **50%**: um preenchimento "de verdade" cobre quase todo o miolo; exigir pelo menos metade dá margem para preenchimentos imperfeitos. - Os dois ficam em constantes no topo de `marcacoes.py`, fáceis de ajustar depois de testar com folhas reais.

8.7 Passo 4: regras de cada questão

```
def status_questao(estados):
    marcados = [i for i, e in enumerate(estados) if e is EstadoCelula.MARCADO]
    duvidas = sum(e is EstadoCelula.DUVIDA for e in estados)
    if len(marcados) >= 2:
        return StatusQuestao.ANULADA_MULTIPLA, None
    if duvidas:
        return StatusQuestao.ANULADA_AMBIGUA, None
    if marcados:
        return StatusQuestao.RESPONDIDA, ALTERNATIVAS[marcados[0]]
    return StatusQuestao.EM_BRANCO, None
```

O que tem na linha A-D	Resultado	Vale ponto?
exatamente 1 marcado, nenhuma dúvida	respondida (a letra)	se for igual à oficial
nada marcado, nenhuma dúvida	em branco	não
2 ou mais marcados	anulada: marcou mais de uma	não
alguma dúvida (com 0 ou 1 marcado)	anulada: marcação incompleta	não

A ordem das regras importa. "Marcação múltipla" é testada antes de "dúvida" porque é a explicação mais forte: se o aluno pintou A e C inteiros, o motivo principal é ter marcado duas, mesmo que também haja um rabisco em B.

Decisão do grupo: uma dúvida **anula** a questão mesmo que outra alternativa esteja bem pintada. Exemplo: A pintado e um X em C. Não dá para saber se o X foi uma "desmarcação" ou uma segunda resposta; na dúvida, anula.

Folha de Respostas
Tópicos em Tecnologia da Informação

Preencha **TODO** o quadrado com caneta preta ou azul.
Marque apenas uma alternativa por questão. Não rasure.

Nome: **JOAO PEDRO**

CPF: **123.456.789-09** RG: **12.345.678-9**

	A	B	C	D
1.	100%			
2.			100%	
3.				100%
4.		100%		100%
5.	15%			
6.				
7.		100%		
8.			33%	

Não dobre a folha. Na foto, os quatro quadrados dos cantos precisam aparecer.

Resultado final na foto de exemplo: verde = marcado, laranja = dúvida, cinza = vazio, com o % de cada quadrado.

8.8 Limitações conhecidas

Situação	O que acontece
X ou tique muito fino	Pode ficar abaixo de 15% e ser lido como em branco em vez de anulado
Lápis ou caneta muito clara	Pode não entrar na máscara de tinta; lido como em branco
Rabisco fora do quadrado	Não é contado (só medimos dentro)
Corretivo branco em cima de uma marcação	Funciona: o corretivo é claro e vira "papel"

OCR: lendo Nome, CPF e RG

OCR (*Optical Character Recognition*, reconhecimento óptico de caracteres) é transformar a **imagem** de um texto em **texto** de verdade (uma *string*). Das etapas do projeto, é a única que usa **aprendizado de máquina**: todo o resto (capítulos 5 a 8) são regras e contas que escrevemos.

Teoria

9.1 Por que texto escrito à mão é difícil

Em um **quadrado de resposta** a pergunta é simples: "tem tinta ou não?". Uma letra é muito mais variada:

Dificuldade	Exemplo
Cada pessoa escreve de um jeito	O "a" de duas pessoas pode não se parecer em nada
Letras parecidas	0 × 0, I × l × 1, S × 5, B × 8
Cursiva: letras emendadas	Em "Maria" cursiva, onde termina o "a" e começa o "r"? O computador não tem como "recortar" letra por letra
Tamanho e inclinação variam	Mesmo dentro de uma palavra

Por isso não dá para resolver com regras escritas à mão ("se tem uma barra vertical e um arco, é um P..."). A solução moderna é **treinar uma rede neural** com milhões de exemplos de texto.

9.2 Ponte com a aula de filtros: redes convolucionais

Na aula de filtros aplicamos **convolução** com máscaras **escolhidas por nós** (média, Sobel, Laplaciano) para realçar bordas e texturas. Uma **rede neural convolucional (CNN)** faz a mesma operação, só que os **valores das máscaras são aprendidos** durante o treino. As primeiras camadas aprendem detectores de bordas e traços; as seguintes combinam traços em curvas e partes de letras.

9.3 Como o EasyOCR funciona

O EasyOCR trabalha em **duas etapas**, cada uma com uma rede neural:



1. Detecção: CRAFT (*Character Region Awareness For Text detection*). A rede olha a imagem inteira e produz, para cada pixel, duas "notas": - **região**: o quanto aquele pixel parece estar **no meio de uma letra**; - **afinidade**: o quanto parece estar **entre duas letras da mesma palavra**.

Juntando as áreas com notas altas, saem as **caixas** de cada palavra ou trecho.

2. Reconhecimento: CRNN (rede convolucional + recorrente). Para cada caixa:

imagem da palavra → CNN (ResNet/VGG) → sequência de "fatias" → LSTM → letra provável por fatia → CTC → texto

1. A **CNN** extrai características da imagem e a transforma numa **sequência de fatias verticais**, da esquerda para a direita.
2. A **LSTM** (rede recorrente) lê essa sequência **considerando o contexto**, ou seja, o que veio antes e depois. Isso ajuda a desempatar: num nome, depois de "MAR" é mais provável "I" que "1".
3. Para cada fatia, a rede dá uma **probabilidade para cada caractere possível**, mais um símbolo especial "vazio" (-).
4. A decodificação **CTC** (*Connectionist Temporal Classification*) transforma a sequência de fatias em texto: **junta repetições seguidas e remove os vazios**:

```
fatias:  M M - A A - R - I I - A -  
junta:  M  - A  - R - I  - A -  
remove: M    A    R    I    A  
texto:  "MARIA"
```

É isso que permite ler uma palavra **sem saber onde cada letra começa e termina**: a rede não precisa recortar as letras, só dizer "o que aparece" em cada fatia.

Confiança: o EasyOCR calcula, a partir das probabilidades que a rede deu aos caracteres escolhidos, um número de 0 a 1 para cada trecho. **Não é garantia de acerto**: é o quanto a rede "tinha certeza".

9.4 Por que a cursiva continua difícil

O modelo do EasyOCR foi treinado **principalmente com texto impresso** (placas, documentos, textos gerados por computador). Letra de forma manuscrita ainda se parece com texto impresso; **cursiva, em geral, não**. Por isso a expectativa, anotada desde o início do projeto, é: **letra de forma razoável, cursiva provavelmente ruim**.

Existem modelos treinados especificamente para manuscrito (como o TrOCR) e modelos de IA de visão (os mesmos por trás de assistentes como o Claude) que leem cursiva muito melhor. **O grupo escolheu o EasyOCR de propósito**: roda no computador, sem internet e sem custo, e deixa a limitação visível para a parte "o que não deu certo" da apresentação.

Como aplicamos

9.5 O código (gabarito/ocr.py)

```
def recortar_campo(folha_bgr, campo):
    m = 8 # descarta a borda da caixa
    recorte = folha_bgr[campo.y + m : campo.y + campo.h - m, campo.x + m : campo.x + campo.w - m]
    cinza = cv2.cvtColor(recorte, cv2.COLOR_BGR2GRAY)
    return cv2.resize(cinza, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC) # amplia 2x

class LeitorOCR:
    def __call__(self, folha_bgr):
        return Identificacao(
            nome=self._ler(folha_bgr, layout.CAMPO_NOME),
            cpf=self._ler(folha_bgr, layout.CAMPO_CPF, allowlist="0123456789.-"),
            rg=self._ler(folha_bgr, layout.CAMPO_RG, allowlist="0123456789.-Xx"),
        )
```

Decisão	Por quê
Recortar da folha alinhada	Como a folha já foi endireitada, a caixa do Nome está sempre no mesmo lugar (layout.CAMPO_NOME). O OCR recebe só o que interessa
Descartar 8 px da borda	A linha da caixa poderia ser lida como "I" ou "l"
Ampliar 2x (bicúbica)	Na folha padrão, a letra tem ~40 px de altura. Ampliar deixa os traços mais "grossos" para a rede, que ignora elementos muito pequenos
Allowlist no CPF/RG	Restringe os caracteres que a rede pode devolver: um 0 vira 0 , um 1 vira 1 . O "X" do RG é permitido (dígito verificador)
Nome sem allowlist	Nomes têm acentos e muitas letras
Juntar os pedaços	O EasyOCR pode devolver "MARIA" e "SILVA" como caixas separadas; ordenamos pela posição x e juntamos com espaço. A confiança do campo é a média dos pedaços
Modelo carregado uma vez só	Carregar a rede leva vários segundos. O LeitorOCR carrega no primeiro uso e o app guarda o objeto com st.cache_resource
Falha no OCR não bloqueia a nota	ler_folha captura qualquer erro do OCR e só mostra um aviso

9.6 Como o app mostra a confiança

Confiança média	Selo no app
≥ 0,70	confiança alta (verde)
0,40 a 0,69	confiança média (laranja)
< 0,40	confiança baixa (vermelho)
nada lido	"Não lido"

9.7 Resultados até agora

Teste	Resultado	Vale como prova?
Fotos sintéticas, texto em Arial (parecido com letra de forma)	100% dos campos corretos	Mostra que o pipeline funciona (recorte, allowlist, junção)

Teste	Resultado	Vale como prova?
Fotos sintéticas, texto em Segoe Script ("cursiva" de computador)	100% dos campos corretos	⚠ Não: é uma fonte digital, regular demais. Letra cursiva de verdade é muito mais difícil
Fotos reais (letra de mão)	<i>a medir: ver resultados.md</i>	✅ Esta é a avaliação que conta

Correção e interface

Este capítulo tem pouca teoria de processamento de imagem. Ele explica como a **leitura** vira uma **nota** e como o aplicativo web funciona.

10.1 O gabarito oficial

O gabarito oficial é, no fim, um dicionário simples: `{1: "A", 2: "C", ..., 8: "B"}`. Ele pode vir de dois lugares (`gabarito/correcao.py`):

Da foto/PDF de uma folha-mestre

A folha-mestre passa **pelo mesmo** `ler_folha` de uma folha de aluno. A diferença é a **validação**: todas as 8 questões precisam ter saído como **respondida**:

```
def gabarito_de_leitura(leitura):
    problemas = [q.numero for q in leitura.questoes if q.status is not
                  StatusQuestao.RESPONDIDA]
    if problemas:
        raise GabaritoInvalido(f"... Verifique as questões: {'', ' '.join(map(str,
        problemas))}.".format(problemas))
    return {q.numero: q.letra for q in leitura.questoes}
```

Por quê? Se a folha-mestre tivesse uma questão em branco ou anulada, não saberíamos a resposta certa dela, e corrigir **todos** os alunos com um gabarito errado é pior que recusar a folha-mestre.

De um arquivo JSON

```
{"1": "A", "2": "C", "3": "B", "4": "D", "5": "A", "6": "A", "7": "C", "8": "B"}
```

Validações feitas, cada uma com mensagem própria:

Problema	Mensagem
Não é JSON	"O arquivo não é um JSON válido."
É uma lista em vez de objeto	"O JSON deve ser um objeto..."
Faltou questão	"Faltam as questões: 8."
Questão que não existe	"Questões que não existem na prova: 9."
Letra inválida	"Questão 3: resposta "E" inválida (use A, B, C ou D)."

Letras minúsculas e espaços são aceitos (`" a "` vira `"A"`).

10.2 A nota

```
def corrigir(oficial, aluno):
    linhas = tuple(
        LinhaResultado(
            numero=q.numero,
            oficial=oficial[q.numero],
            aluno=q,
            correta=q.status is StatusQuestao.RESPONDIDA and q.letra == oficial[q.numero],
        )
        for q in aluno.questoes
    )
    return Resultado(linhas)
```

A questão **só vale ponto** se foi respondida **e** a letra bate com a oficial. Em branco e anulada valem **zero**. A anulação é da **resposta do aluno**, não da questão para a turma toda.

O `Resultado` calcula quatro contagens, que sempre somam 8: **certas, erradas, em branco e anuladas**.

10.3 Como o Streamlit funciona

O **Streamlit** transforma um script Python em página web. O modelo mental é diferente de um site comum e explica várias decisões do `app.py`:

A cada clique ou upload, o Streamlit roda o `app.py` inteiro de novo, de cima para baixo.

Isso traz três consequências:

1. Variáveis comuns "esquecem" tudo a cada interação. Para lembrar o gabarito oficial entre uma execução e outra, ele fica em `st.session_state`, uma espécie de dicionário que sobrevive às reexecuções (enquanto a aba estiver aberta):

```
estado = st.session_state
estado.setdefault("oficial", None)    # só define na primeira vez
...
estado.oficial = gabarito_de_leitura(leitura_mestre)
```

2. Sem cuidado, a mesma foto seria processada a cada clique. Por isso as leituras usam **cache**:

```
@st.cache_data(show_spinner=False, max_entries=20)
def ler_aluno(dados: bytes):
    return ler_folha(carregar_imagem(dados), ocr=leitor_ocr())

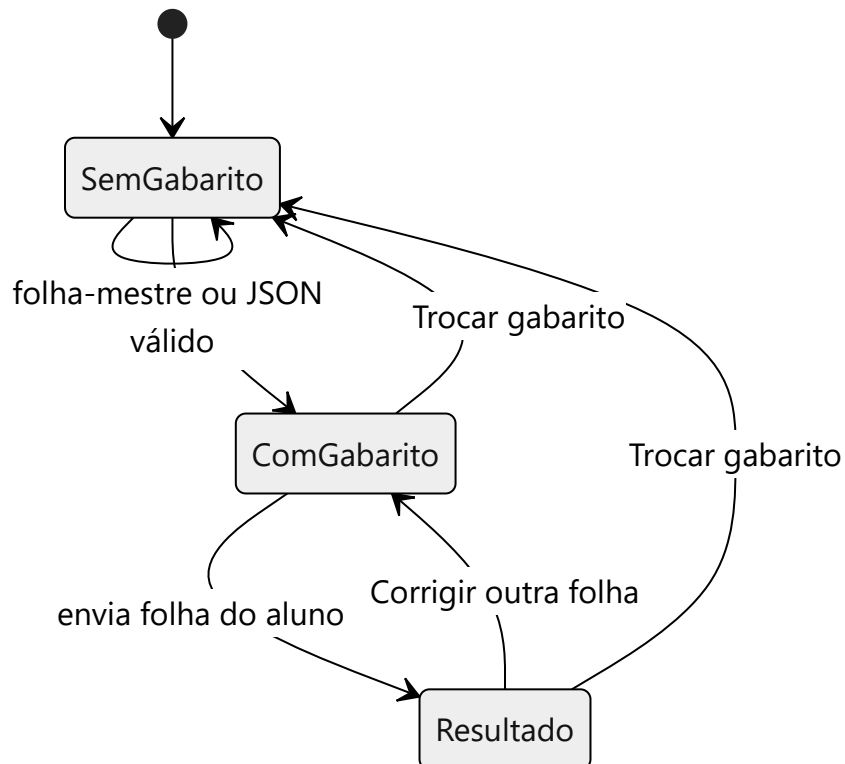
@st.cache_resource(show_spinner=False)
def leitor_ocr():
    return LeitorOCR()
```

- `cache_data`: se os **bytes do arquivo** forem os mesmos, devolve o resultado guardado sem processar de novo.
- `cache_resource`: guarda **um único objeto** para o app todo; aqui, o modelo do OCR, que é pesado.

3. Como "limpar" um campo de upload? Um widget do Streamlit é identificado pela sua `key`. Para o botão "Corrigir outra folha" esvaziar o upload, trocamos a chave:

```
arquivo_aluno = st.file_uploader(..., key=f"aluno_{estado.versao_aluno}")
...
if st.button("Corrigir outra folha"):
    estado.versao_aluno += 1    # aluno_0 → aluno_1: é um widget novo, vazio
    st.rerun()
```

O fluxo de telas



10.4 A tela de resultado

O resultado é um HTML gerado por `gabarito/interface_html.py` e estilizado por `estilo.css`:

- **Placar** grande: "6 de 8", e o resumo "6 certas, 1 errada, 1 anulada".
- **Identificação**: Nome, CPF e RG com o selo de confiança do OCR.
- **Réplica da folha**: 8 linhas com as bolinhas A-D:
- **contorno escuro** = resposta oficial;
- **bolinha preenchida** = marcada pelo aluno; **verde** se certa, **vermelha** se errada, **laranja** se anulada;
- **listrada em laranja** = marcação em dúvida.
- **Ver processamento**: as abas *Grade detectada* (capítulo 8), *Folha alinhada* (capítulo 6) e *Arquivo original*.

Dois cuidados técnicos: - O HTML é gerado por uma função pura, que recebe o `Resultado` e devolve texto, sem depender do Streamlit. Por isso ela tem testes automáticos (`tests/test_interface_html.py`). - Texto lido pelo OCR é "escapado" (`html.escape`) antes de entrar no HTML. Se alguém escrevesse `<b>` no campo Nome e o OCR lesse isso, sem o escape o navegador interpretaria como código. É uma proteção padrão de segurança web (evita *injeção de HTML*).

10.5 A imagem "grade detectada"

`gabarito/visualizacao.py` desenha, em cima da folha alinhada, um retângulo em cada quadrado **na posição encontrada pelo refinamento** (não na posição do template), com a cor do estado e o % de tinta:

```
CORES_RGB = {
    EstadoCelula.MARCADO: (52, 199, 89),    # verde
    EstadoCelula.DUVIDA: (255, 159, 10),    # laranja
    EstadoCelula.VAZIO: (174, 174, 178),    # cinza
}
```

É a figura mais útil para **explicar um erro de leitura**: dá para ver exatamente o que o programa "achou" de cada quadrado.

10.6 O visual

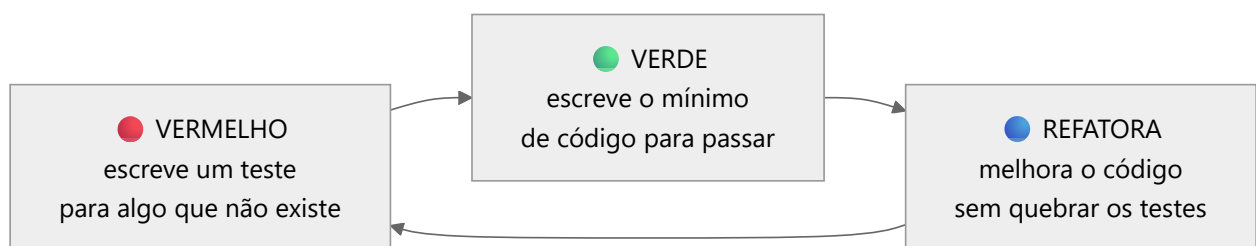
O grupo pediu um visual no estilo Apple: fundo cinza-claro `#F5F5F7`, grupos brancos com cantos arredondados, fonte do sistema, bastante espaço em branco e cores de status do sistema da Apple (verde `#34C759`, vermelho `#FF3B30`, laranja `#FF9F0A`). As mesmas cores são usadas na interface e na imagem da grade detectada, para as duas "falarem a mesma língua".

Testes e validação

"Funcionou quando eu testei" não é prova. Este capítulo explica **como sabemos** que o software funciona e, tão importante quanto, **o que ainda não foi provado**.

11.1 TDD: o teste vem antes do código

O projeto foi escrito com **TDD** (*Test-Driven Development*), num ciclo de três passos:



Por que escrever o teste antes? Um teste escrito depois do código tende a testar "o que o código faz", e não "o que ele deveria fazer". Ver o teste **falhar primeiro** prova que ele realmente detecta a ausência da funcionalidade.

Exemplo real do projeto: o suporte a PDF. 1. ● Escrevemos

`test_carregar_pdf_renderiza_a_pagina_e_os_marcadores_sao_encontrados` e rodamos: **falhou** com "Não consegui abrir o arquivo como imagem". 2. ● Implementamos `_renderizar_pdf` com o `pypdfium2`: **passou**. 3. Rodamos a suíte inteira para garantir que nada mais quebrou.

11.2 O que é testado

```

.\.venv\Scripts\python.exe -m pytest          # 88 testes, ~10 s
.\.venv\Scripts\python.exe -m pytest -m ocr    # +1 teste com o EasyOCR de verdade
  
```

Arquivo	Testes	O que garante
<code>test_layout.py</code>	9	Quadrados dentro da página, janelas de busca sem sobreposição, nada na zona de silêncio dos marcadores
<code>test_folha.py</code>	6	A folha gerada tem tamanho A4 e os 4 ArUco são detectados na posição exata do layout
<code>test_alinhamento.py</code>	8	Orientação EXIF, arquivo inválido, PDF (inclusive com várias páginas e corrompido), folha de cabeça para baixo, erro que nomeia os cantos faltando

Arquivo	Testes	O que garante
<code>test_marcacoes.py</code>	20	Faixas vazio/dúvida/marcado, todas as regras de anulação, caneta azul, sombra forte, papel liso sem tinta, quadrados impressos a 85% e 115%, quadrado deslocado
<code>test_leitura.py</code>	16	Fotos sintéticas de ponta a ponta em 10 cenários, 3 variações de tamanho de impressão, PDF, e o OCR falhando sem derrubar a correção
<code>test_correcao.py</code>	10	JSON válido/inválido, folha-mestre com problemas, contagem de certas/erradas/brancos/anuladas
<code>test_ocr.py</code>	10	Recorte sem a borda, junção de pedaços, níveis de confiança (+1 teste com o modelo real)
<code>test_interface_html.py</code>	8	Placar, resumo, classe de cada linha, bolinhas oficial/aluno, escape de HTML
<code>test_visualizacao.py</code>	2	Cores da grade detectada

11.3 Fotos sintéticas: como testar sem imprimir 500 folhas

Para testar "foto torta, com sombra, de cabeça para baixo" precisaríamos de muitas fotos reais, **com resposta conhecida**. A solução foi **simular** essas fotos (`gabarito/sintetico.py`).

Passo 1: preencher a folha no computador (`folha_preenchida`)

Desenha marcações na folha gerada:

Tipo	Como é desenhado
<code>cheio</code>	Polígono com cantos irregulares + zigue-zague de "caneta" (imita preenchimento à mão)
<code>x</code>	Duas diagonais
<code>traco</code>	Uma linha horizontal
<code>parcial</code>	Só a parte esquerda do quadrado (~1/3 da região medida)

Também pode **redesenhar os quadrados com tamanhos aleatórios** (ex.: entre 85% e 115%), simulando impressão torta, e escrever Nome/CPF/RG com uma fonte.

Passo 2: "fotografar" (`fotografar`)

Cada efeito imita um problema real, **usando as técnicas dos capítulos anteriores ao contrário**:

Efeito	Como é simulado	Problema real que imita
Escala	<code>cv2.resize</code> com <code>INTER_AREA</code>	Distância da câmera
Perspectiva	Desloca os 4 cantos aleatoriamente e aplica uma homografia	Celular inclinado
Rotação	Gira os cantos com uma matriz de rotação	Folha torta na mesa
Fundo	<code>borderValue</code> marrom-escuro no <code>warpPerspective</code>	Mesa
Sombra	Multiplica por um degradê linear (de 1 até 1 – intensidade)	Mão ou corpo fazendo sombra
Desfoque	Filtro gaussiano	Foco ruim, tremida
Ruído	Soma ruído gaussiano	Sensor de celular com pouca luz

Efeito	Como é simulado	Problema real que imita
JPEG	Codifica e decodifica com qualidade 88	Compressão da câmera

Os cenários testados

Cenário	Parâmetros
reta	nenhum efeito além da escala
girada 5° / girada 30°	rotação
de cabeça para baixo	rotação de 180°
perspectiva	cantos deslocados até 8% da largura
sombra	degradê até 50%
sombra forte	degradê até 75% + rotação + perspectiva
desfocada e ruidosa	desfoque $\sigma = 1,5$ + ruído
pequena na foto	folha ocupando pouco da imagem (escala 0,35)
tudo junto	rotação, perspectiva, sombra, desfoque e ruído ao mesmo tempo

Em todos, as 8 questões precisam ser lidas **exatamente** como esperado: 5 respondidas (preto e azul), 1 marcação dupla, 1 X, 1 em branco e 1 parcial.

11.4 Experimentos: provando que cada parte é necessária

Um bom jeito de entender (e de mostrar que se entendeu) é **desligar uma parte do algoritmo e ver o que quebra**. O script `scripts/experimentos.py` faz isso **sem alterar nenhum arquivo**:

```

.\.venv\Scripts\python.exe scripts\experimentos.py           # lista os experimentos
.\.venv\Scripts\python.exe scripts\experimentos.py sem_refinamento # roda um

```

Resultados verificados:

Experimento	O que desliga	O que quebra	Lição
<code>sem_refinamento</code>	Procurar o quadrado real	<code>test_refinamento_acompanha_quadrado_deslocado</code>	Sem refinamento, um quadrado impresso fora do lugar é medido pela metade
<code>sem_margem_interna</code>	Ignorar a borda do quadrado	20 testes , inclusive a folha em branco	A borda impressa conta como tinta: quadrados vazios parecem "em dúvida"
<code>limite_vazio_30</code>	Sobe o limite de vazio para 30%	16 testes	O X (24%) passa a ser lido como em branco, e a questão deixa de ser anulada

Experimento	O que desliga	O que quebra	Lição
<code>sem_dilatacao</code>	A dilatação na estimativa do fundo	3 testes com quadrados pintados	Sem dilatação, o "fundo" inclui a própria tinta; quadrado pintado ÷ fundo escuro = claro, e a marcação some
<code>sem_trava_do_limiar</code>	Prender o Otsu entre 100 e 200	<code>test_papel_liso_com_ruído_nao_vira_tinta</code>	Numa região sem tinta, o Otsu separa ruído de papel e inventa tinta
<code>otsu_direto</code>	Toda a compensação de sombra	Cenário sombra forte + 2 testes de sombra/papel	Uma sombra forte vira "tinta" (figura 9 do capítulo 7)

Transparência: dois desses testes (papel liso e sombra forte) foram **adicionados enquanto escrevíamos esta documentação**. Ao rodar os experimentos, percebemos que desligar a trava do limiar não quebrava nenhum teste, e que o teste de sombra antigo usava uma sombra fraca demais para exigir a compensação. Ou seja, essas partes do código funcionavam, mas **não estavam protegidas**. Rodar experimentos assim é uma boa forma de achar buracos nos testes.

11.5 Teste manual no navegador

Além do pytest, o app foi usado de verdade num navegador automatizado (Playwright), verificando: - gabarito por foto, por JSON e por PDF; - folha-mestre inválida (mensagem com as questões a conferir); - alunos com anuladas, com erros, de cabeça para baixo, em PDF; - botão "Corrigir outra folha"; - tela em largura de celular (400 px), sem rolagem horizontal; - console do navegador sem erros.

Problemas encontrados **só** nesse teste manual, e corrigidos: - a lista de questões aparecia deslocada para a direita (o CSS do Streamlit aplicava um recuo em listas); - em questões anuladas, não dava para ver qual era a resposta oficial (a cor laranja cobria o contorno); - depois de mudar o código, o app continuava rodando a versão antiga (é preciso reiniciar o Streamlit).

11.6 Avaliação com amostras (`scripts/avaliar.py`)

Roda o pipeline completo, **com OCR**, em todas as imagens de uma pasta e compara com o `.json` esperado de cada uma:

```
..\venv\Scripts\python.exe scripts\avaliar.py amostras\sinteticas
```

Resultado nas amostras sintéticas: **48/48 respostas e 15/15 campos de OCR** corretos (5 fotos + 1 PDF).

11.7 O que ainda não foi provado

Não provado	Por quê	Como provar
Funciona com fotos reais de celular	Toda a validação automática usa fotos simuladas	Fotografar folhas reais e rodar o <code>avaliar.py</code> em <code>amostras/reais/</code>
OCR em letra de mão real , principalmente cursiva	Os testes usam fontes de computador	Idem, com nomes escritos à mão
Limites de 15% e 50% ideais para canetas reais	Escolhidos por raciocínio e fotos sintéticas	Olhar os % na grade detectada das fotos reais e ajustar se preciso
Marcação muito fraca, lápis, papel amassado	Não simulados	Testar e registrar em resultados.md

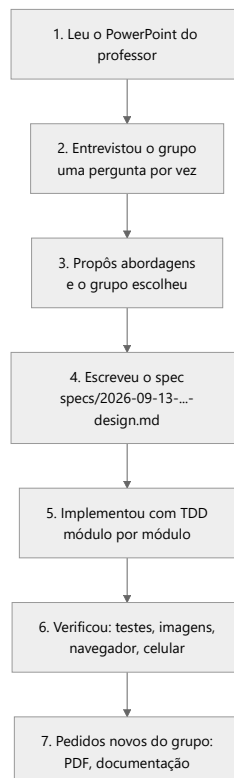
Como a IA foi usada

O professor liberou o uso de IA, com uma condição: **o grupo precisa entender o que a IA fez**. Este capítulo registra, com honestidade, **quem decidiu o quê, o que a IA produziu e como conferir**.

12.1 Ferramenta

- **Claude Code** (assistente de programação da Anthropic, modelo Claude Opus 5), rodando no terminal, com acesso à pasta do projeto.
- A IA consultou a **documentação atual** das bibliotecas (OpenCV, Streamlit, EasyOCR, pypdfium2) antes de usá-las, em vez de confiar só na memória. Isso importou: por exemplo, o pip instalou o **OpenCV 5.0**, e foi preciso confirmar que a API de ArUco continuava igual.
- Para testar a interface, a IA controlou um navegador automatizado (**Playwright**), enviando arquivos e tirando capturas de tela.

12.2 O processo, em ordem



1. Leitura do enunciado

Um `.pptx` é um arquivo ZIP com XMLs dentro. A IA extrai o texto dos slides e as imagens (as duas folhas de exemplo preenchidas com rabiscos azul e preto) para entender o problema.

2 e 3. Entrevista e decisões

A IA fez perguntas de múltipla escolha, sempre indicando uma recomendação. **Várias decisões foram diferentes da recomendação da IA**, e isso é importante registrar:

Pergunta	Recomendação da IA	Decisão do grupo
Linguagem	Python + OpenCV (mais documentação para OMR, OCR de manuscrito, gratuito)	Aceitou Python + OpenCV
Como a imagem chega	—	Foto de celular (arquivo) ; depois o grupo pediu também PDF
Gabarito oficial	Foto de uma folha-mestre	Folha-mestre OU arquivo JSON
OCR do nome	IA de visão via API (melhor em cursiva)	Tesseract/EasyOCR (local; a IA escolheu o EasyOCR entre os dois por instalar só com pip)
CPF e RG	—	OCR, como o nome
Interface	App web local (Streamlit)	Streamlit, com fluxo simplificado : gabarito mestre → aluno → nota
Marcação fraca	3 faixas (vazio/dúvida/marcado)	Aceitou
Folha	Gerada por código	Aceitou
Localizar a folha	Marcadores ArUco (abordagem A)	Aceitou (contra B: quadrados pretos, e C: sem marcadores)
Regras	Anulada vale 0; dúvida anula	Aprovou
Visual	—	Estilo Apple , pedido pelo grupo

Depois, o grupo testou a **folha original do professor** (sem marcadores) e ela foi recusada. A IA propôs um segundo modo de leitura, sem marcadores, para prints; **o grupo decidiu não fazer** e usar a folha do software.

4. Spec

Antes de programar, a IA escreveu o documento de design com todas as decisões, arquitetura, regras e testes planejados: [specs/2026-09-13-corretor-gabarito-design.md](https://github.com/mauricioalmeida/2026-09-13-corretor-gabarito-design.md).

5. Implementação

Na ordem: layout → folha → marcacoes → alinhamento + sintetico + leitura → correcao → ocr → visualizacao → interface_html → app.py → scripts → PDF → documentação. Cada módulo seguiu o ciclo TDD do [capítulo 11](#).

6. Problemas encontrados no caminho

Uma IA também erra. O que importa é o processo pegar os erros. Os que apareceram:

Problema	Como foi percebido	Correção
As janelas de busca de linhas vizinhas se sobrepunham	Teste de layout falhou	Aumentou o passo entre linhas
A moldura da grade e o rodapé encostavam no marcador inferior direito, que deixou de ser detectado	Teste da folha falhou	Novo layout + teste da "zona de silêncio"
A marcação sintética "metade" daria exatamente 50%, a fronteira entre dúvida e marcado	Conta feita antes de rodar o teste	Trocada por "parcial" (~33%)
<code>warpPerspective</code> não aceita <code>INTER_AREA</code>	Revisão do código	Superamostragem: gerar 2× e reduzir
<code>opencv-python</code> e o EasyOCR (que usa <code>opencv-python-headless</code>) conflitam	Conhecimento prévio, antes de instalar	Só o <code>headless</code> no <code>requirements.txt</code>
Lista de questões deslocada na tela	Captura de tela no navegador	CSS mais específico
Resposta oficial invisível em questões anuladas	Captura de tela	Anel escuro em volta da bolinha
App rodando código velho depois da mudança para PDF	Mensagem de erro antiga no navegador	Reiniciar o servidor (documentado)
A trava do limiar e a compensação de sombra não eram realmente testadas	Experimentos de "desligar partes"	Dois testes novos
OCR 100% nas amostras poderia ser mal interpretado	Análise dos dados	Aviso explícito: fontes de computador não são letra de mão

12.3 O que é do grupo (e não da IA)

- **As decisões** da tabela acima.
- **Os testes com folhas reais**, fotos e o preenchimento de [resultados.md](#). A IA não tem como fotografar papel.
- **A apresentação**: explicar o funcionamento com as próprias palavras.

12.4 Como conferir que o grupo entendeu

Um roteiro prático, do mais simples ao mais profundo:

1. **Rodar tudo**: app, testes e `gerar_amostras.py` ([capítulo 1](#)).
2. **Explicar a figura de cada etapa** em [02-visao-geral.md](#) sem ler o texto.
3. **Refazer as contas à mão**: - a posição de `celula(5, "B")` ([capítulo 4](#)); - o Otsu do exemplo de 10 pixels ([capítulo 7](#)); - o preenchimento de 1000 pixels de tinta num miolo de 44 × 44 ([capítulo 8](#)).
4. **Rodar os experimentos** do capítulo 11 e explicar **por que** cada teste quebrou.
5. **Mudar um parâmetro de propósito** (por exemplo `LIMITE_MARCADO = 0.8` em `gabarito/marcacoes.py`), rodar o app com uma foto e explicar o que mudou na grade detectada. Depois, desfazer a mudança.
6. **Responder** as [perguntas e respostas](#) sem olhar.

12.5 Mapa: conceito da disciplina → onde está no código

Conceito visto em aula	Onde aparece
Imagem como matriz, canais RGB	<code>carregar_imagem</code> , <code>etapas_mascara (folha_bgr.min(axis=2))</code>
Conversão para tons de cinza	<code>alinhar</code> (detecção ArUco), <code>recortar_campo</code> (OCR)
Limiarização / binarização	<code>etapas_mascara</code> (Otsu)
Filtro gaussiano (suavização)	<code>etapas_mascara</code> (fundo), <code>sinetico.fotografar</code> (desfoque)
Convolução	Filtro gaussiano; as CNNs do EasyOCR (capítulo 9)
Morfologia (dilatação)	<code>etapas_mascara</code> (estimativa do fundo)
Ruído	<code>sinetico.fotografar</code> (ruído gaussiano); robustez testada
Transformações geométricas	<code>alinhar</code> (homografia), <code>sinetico.fotografar</code> (rotação, perspectiva)
Interpolação	<code>warpPerspective</code> (bilinear), <code>resize</code> (área e bicúbica)
Segmentação e contornos	<code>localizar_quadrado (findContours, boundingRect)</code>

Perguntas e respostas

Perguntas que o professor ou a turma podem fazer, com respostas curtas e o capítulo onde está a explicação completa.

Sobre a abordagem

Por que vocês não usaram a folha do professor? O layout era livre. Nossa folha tem **marcadores ArUco** nos cantos, que permitem achar e endireitar a folha em qualquer foto. A folha original não tem nenhuma referência desse tipo, e localizar a grade nela seria bem menos confiável. → [Cap. 4, 5](#)

Por que ArUco e não 4 quadrados pretos? Quadrados pretos são todos iguais: não dizem **qual** canto é qual. Com a folha girada 180° o programa se confundiria. Cada ArUco tem um **ID** e uma orientação própria. → [Cap. 5.6](#)

Por que homografia e não só girar a imagem? Rotação não desfaz **perspectiva** (celular inclinado). A homografia corrige rotação, escala, deslocamento e perspectiva de uma vez, e é a transformação certa para a foto de um plano. → [Cap. 6](#)

Por que exatamente 4 marcadores? A homografia tem 8 incógnitas; cada ponto dá 2 equações; 4 pontos dão 8 equações. → [Cap. 6.4](#)

E se a folha estiver de cabeça para baixo? Funciona. O ID de cada marcador diz qual canto da folha ele é, independentemente de como aparece na foto. → [Cap. 5.4](#)

Sobre a leitura

Como o programa sabe se um quadrado está pintado? Mede a **fração de pixels de tinta** na parte interna do quadrado: até 15% é vazio, a partir de 50% é marcado, e o que fica entre os dois é dúvida. → [Cap. 8](#)

Por que medir só a parte de dentro do quadrado? Porque a **borda impressa** também é preta. Medindo o quadrado inteiro, todo quadrado vazio pareceria parcialmente pintado; o experimento `sem_margem_interna` quebra 20 testes. → [Cap. 8.5](#)

Por que não usar só a imagem em tons de cinza? Na conversão comum o azul pesa pouco, e caneta azul clara vira cinza médio. Usamos o **menor canal (R, G ou B)**: papel é claro em todos, tinta é escura em pelo menos um. → [Cap. 7.1](#)

O que é o método de Otsu? Um jeito automático de escolher o limiar entre escuro e claro: testa todos os cortes e fica com o que **maximiza a variância entre as duas classes**. → [Cap. 7.5](#)

Como vocês tratam sombra? Estimamos como seria o papel sem tinta (dilatação com janela maior que qualquer mancha de tinta + desfoque) e **dividimos** a imagem por essa estimativa. A sombra multiplica tudo por um fator, e a divisão cancela esse fator. → [Cap. 7.3](#)

E se os quadrados forem impressos com tamanhos diferentes? Dois mecanismos: o **refinamento local** procura o quadrado real perto da posição esperada, e a **região interna** medida é pequena o bastante para ficar dentro de quadrados entre 85% e 115% do tamanho. Há testes para os dois. → [Cap. 8.4 e 8.5](#)

O que acontece se o aluno marcar duas alternativas? A questão é **anulada** e vale 0; o app mostra "Anulada: marcou mais de uma". → [Cap. 8.7](#)

E se ele fizer um X em vez de pintar? Um X ocupa ~24% do miolo, cai na faixa de dúvida e a questão é anulada por "marcação incompleta". Um X muito fino pode ficar abaixo de 15% e ser lido como em branco, uma limitação conhecida. → [Cap. 8.6 e 8.8](#)

E se ele marcar A e fizer um rabisco leve em C? O rabisco, se passar de 15%, é "dúvida", e a questão é anulada. Foi uma decisão do grupo: não dá para saber se o rabisco era uma segunda resposta. → [Cap. 8.7](#)

Por que 15% e 50%? 15% fica bem acima de um quadrado vazio (0%) e abaixo de um X normal; 50% exige um preenchimento de verdade com margem para imperfeições. São constantes fáceis de ajustar depois dos testes com fotos reais. → [Cap. 8.6](#)

Sobre o OCR

Como o programa lê o nome? Recorta a caixa do Nome da folha já alinhada e passa para o **EasyOCR**, que usa duas redes neurais: uma acha onde há texto (CRAFT) e outra reconhece os caracteres (CNN + LSTM + CTC). → [Cap. 9](#)

Funciona com letra cursiva? Esperamos que **não muito bem**: o modelo foi treinado principalmente com texto impresso, e em cursiva as letras são emendadas. É uma limitação que assumimos e medimos com fotos reais. → [Cap. 9.4, resultados](#)

Por que não usaram uma IA mais forte para ler a cursiva? Foi considerado (a IA recomendou), mas o grupo escolheu uma solução **local, gratuita e sem internet**. O preço é a cursiva mais fraca, e isso fica explícito na apresentação. → [Cap. 12.2](#)

O que acontece se o OCR falhar? A nota sai normalmente; só a identificação fica como "Não lido" ou aparece um aviso. → [Cap. 2.4](#)

Sobre o software

Por que Python e não MATLAB, como nas aulas? O professor liberou a escolha. Python + OpenCV tem muito material sobre OMR, bibliotecas gratuitas de OCR para texto manuscrito e roda sem licença. As operações são as mesmas vistas em aula (limiarização, morfologia, filtros, transformações). → [Cap. 12.5](#)

Como vocês sabem que funciona? 88 testes automáticos, incluindo 10 cenários de foto simulada (girada, com sombra, desfocada, de cabeça para baixo...), PDF e impressão com quadrados de tamanhos diferentes, além de testes manuais no navegador. **Com fotos reais**, o resultado está em [resultados.md](#). → [Cap. 11](#)

Aceita PDF? Sim. A primeira página é convertida em imagem a 200 DPI e segue o mesmo caminho de uma foto. → [Cap. 3](#)

Quanto tempo leva para corrigir uma folha? 1–3 segundos. A primeira correção depois de abrir o app leva ~20–30 s, porque o modelo do OCR é carregado na memória. → [Cap. 1.3](#)

O que a IA fez e o que vocês fizeram? A IA entrevistou o grupo, escreveu o spec, o código, os testes e esta documentação. O grupo tomou as decisões de projeto (várias diferentes da recomendação da IA), testou com folhas reais e é responsável por entender e apresentar. → [Cap. 12](#)

Roteiro da apresentação

Antes de começar - `streamlit run app.py` já aberto no navegador (e o OCR "aquecido": corrija uma folha qualquer uma vez, porque o primeiro carregamento do modelo leva alguns segundos). - Fotos reais em `amostras/reais/` e, como reserva, as sintéticas em `amostras/sinteticas/`.

1. A folha (1 min)

- Mostre a folha impressa: os **4 marcadores ArUco** nos cantos e a instrução "preencha todo o quadrado".
- Explique: cada marcador tem um número, então o programa sabe qual canto é qual, mesmo com a folha girada.

2. Gabarito oficial (1 min)

- Envie a foto da folha-mestre e aponte as pílulas com as 8 respostas.
- Mencione a alternativa por arquivo JSON.

3. Aluno normal (2 min)

- Envie a foto e mostre o placar e a réplica da folha (contorno = oficial, preenchida = aluno).
- Abra **Ver processamento**: 1. *Foto original*: torta, com fundo. 2. *Folha alinhada*: homografia a partir dos 4 cantos. 3. *Grade detectada*: % de tinta por quadrado (verde = marcado, laranja = dúvida, cinza = vazio).

4. Casos de anulação (1 min)

- Marcação dupla → "Anulada, marcou mais de uma".
- X ou meio preenchido → "Anulada, marcação incompleta" (entre 15% e 50% de tinta).

5. Robustez (1 min)

- Foto girada ou de cabeça para baixo, com sombra → mesma nota.
- PDF escaneado → mesma leitura.
- Quadrados impressos com tamanhos diferentes → o programa procura o quadrado real perto da posição esperada.

6. Nome, CPF e RG (1 min)

- Letra de forma vs cursiva: compare o texto lido e o selo de confiança.

7. O que deu certo e o que não deu (2 min)

- Use a tabela de [resultados.md](#).

Para estudar cada etapa em detalhe, veja a [documentação completa](#). As perguntas mais prováveis estão em [13-perguntas-e-respostas.md](#).

Técnicas de processamento de imagem usadas

Etapa	Técnica
Localizar a folha	Marcadores ArUco (OpenCV)
Endireitar	Homografia + <code>warpPerspective</code>
Remover sombra	Dilatação morfológica para estimar o fundo + divisão
Separar tinta do papel	Menor canal RGB + limiarização de Otsu
Achar cada quadrado	Contornos (<code>findContours</code>) na janela de busca
Decidir marcação	Proporção de pixels de tinta na região interna
Ler nome	OCR com rede neural (EasyOCR)

Resultados

1. Fotos sintéticas (automáticas)

Gerado por `python scripts/avaliar.py amostras/sinteticas`. O detalhe está em [resultados_sinteticas.md](#).

- **Respostas:** 48/48 lidas corretamente (5 fotos com rotação de até 180°, perspectiva, sombra, desfoque, ruído e quadrados impressos entre 85% e 115% do tamanho, mais 1 PDF).
- **Suíte de testes:** 88 testes automatizados passando (`pytest`) + 1 smoke test do OCR real (`pytest -m ocr`), com 10 cenários de foto sintética, 3 variações de tamanho de impressão e PDF.
- **OCR:** 15/15 campos lidos 100% iguais ao esperado.

⚠ **Limite desta avaliação:** nas fotos sintéticas, Nome/CPF/RG são escritos com **fontes de computador** (Arial e Segoe Script). A "cursiva" sintética é muito mais regular que a letra de mão real, então o resultado de OCR acima **não** mede letra manuscrita. Isso só aparece com fotos reais (seção 2).

2. Fotos reais (preencher antes da apresentação)

1. Imprima `amostras/folha_de_respostas.pdf` (ou baixe pelo app) e preencha ~8–10 folhas à mão: - uma **folha-mestre** bem preenchida; - alunos com nome em **letra de forma** e em **cursiva**; - casos ruins de propósito: marcação dupla, X, traço, meio preenchido, caneta azul fraca, folha amassada.
2. Fotografe com o celular (ou escaneie em PDF) e copie para `amostras/reais/`.
3. Para cada foto `x.jpg`, crie `x.json` com o esperado, por exemplo: `json {"respostas": {"1": "A", "2": "ANULADA_MULTIPLA", "3": "EM_BRANCO", "4": "C", "5": "D", "6": "B", "7": "ANULADA_AMBIGUA", "8": "A"}, "nome": "Maria Eduarda Lima", "cpf": "987.654.321-00", "rg": "98.765.432-1"}`
4. Rode `python scripts/avaliar.py amostras/reais --saida docs/resultados_reais.md` e resuma aqui.

Situação	Resultado nas fotos reais	Deu certo?
Marcação bem preenchida (preta/azul)		
Marcação dupla → anulada		
X / traço / meio preenchido → anulada ou em branco		
Foto torta, girada ou com sombra		
Nome em letra de forma		
Nome em cursiva		

3. Limitações conhecidas (esperadas)

- **Cursiva:** o EasyOCR foi treinado principalmente com texto impresso. Em letra de mão cursiva real é esperado que erre bastante ou devolva confiança baixa (o app mostra "confiança baixa").
- **Marcador cortado ou coberto:** sem os 4 cantos a folha não é alinhada, e o app pede outra foto.
- **Marca muito leve** (lápiz, caneta falhando) pode ficar abaixo de 15% e ser lida como em branco. Os limites ficam em `gabarito/marcacoes.py` (`LIMITE_VAZIO` , `LIMITE_MARCADO`).
- **Um X ou traço fino** pode passar abaixo de 15% e contar como em branco em vez de anulada.

Avaliação das amostras sintéticas

Avaliação: amostras/sinteticas

Respostas lidas corretamente: 48/48 (100%)

Foto	Respostas lidas certo	Erros de leitura
00_mestre.jpg	8/8	—
01_aluno_nota_8_letra_de_forma.jpg	8/8	—
02_aluno_cursiva_com_erros.jpg	8/8	—
03_aluno_anuladas.jpg	8/8	—
04_aluno_de_cabeca_para_baixo_impressao_torta.jpg	8/8	—
05_aluno_escaneado.pdf	8/8	—

OCR de Nome, CPF e RG

Foto	Campo	Esperado	Lido	Semelhança	Confiança
01_aluno_nota_8_letra_de_forma.jpg	nome	JOAO PEDRO SANTOS	JOAO PEDRO SANTOS	100%	0.73
01_aluno_nota_8_letra_de_forma.jpg	cpf	123.456.789-09	123.456.789-09	100%	0.78
01_aluno_nota_8_letra_de_forma.jpg	rg	12.345.678-9	12.345.678-9	100%	0.97
02_aluno_cursiva_com_erros.jpg	nome	Maria Eduarda Lima	Maria Eduarda Lima	100%	0.80
02_aluno_cursiva_com_erros.jpg	cpf	987.654.321-00	987.654.321-00	100%	0.86
02_aluno_cursiva_com_erros.jpg	rg	98.765.432-1	98.765.432-1	100%	0.99
03_aluno_anuladas.jpg	nome	CARLOS ALBERTO	CARLOS ALBERTO	100%	0.84
03_aluno_anuladas.jpg	cpf	111.222.333-44	111.222.333-44	100%	0.72
03_aluno_anuladas.jpg	rg	11.222.333-4	11.222.333-4	100%	0.95
04_aluno_de_cabeca_para_baixo_impressao_torta.jpg	nome	ANA BEATRIZ	ANA BEATRIZ	100%	1.00
04_aluno_de_cabeca_para_baixo_impressao_torta.jpg	cpf	555.666.777-88	555.666.777-88	100%	0.97
04_aluno_de_cabeca_para_baixo_impressao_torta.jpg	rg	55.666.777-8	55.666.777-8	100%	1.00

Foto	Campo	Esperado	Lido	Semelhança	Confiança
05_aluno_escaneado.pdf	nome	LUCAS MOREIRA	LUCAS MOREIRA	100%	1.00
05_aluno_escaneado.pdf	cpf	222.333.444- 55	222.333.444- 55	100%	0.75
05_aluno_escaneado.pdf	rg	22.333.444-5	22.333.444-5	100%	0.93